

Journal of Statistical Software

MMMMMM YYYY, Volume VV, Issue II. *doi: 10.18637/jss.v000.i00*

RprobitB: Bayesian Probit Choice Models with Preference Heterogeneity in R

Lennart Oelschläger
Bielefeld University

Dietmar Bauer
Bielefeld University

Abstract

The R package **RprobitB** provides numerically efficient Bayesian estimation of the multinomial probit model for discrete choice data, with a focus on unobserved preference heterogeneity. Heterogeneity is modeled by random coefficients with normal or log-normal mixing distributions and by latent classes, whose number is fixed in advance or inferred from the data by sparse finite or Dirichlet process mixtures. Estimation is based on a Gibbs sampler with data augmentation, which avoids the costly evaluation of the probit likelihood. Besides multinomial choices, the package supports binary, ordered, and ranked responses in cross-sectional and panel data. Posterior inference comprises marginal effects, willingness-to-pay measures, and predicted choice probabilities with credible intervals. Competing specifications are compared by information criteria, approximate leave-one-out cross-validation, and Bayes factors. We describe the model class, the sampler, and the software design, and illustrate the workflow with five empirical data sets.

Keywords: discrete choice, multinomial probit, Bayesian inference, Markov chain Monte Carlo, random coefficients, latent classes, Dirichlet process, R.

1. Introduction

Throughout life, individuals make countless choices: commuters select a mode of transport, consumers a product, and patients a treatment. Companies and policymakers seek to understand which attributes shape such decisions and how a modification of these attributes influences choice behavior. The analysis of choice behavior is therefore pervasive in applied fields such as transportation (Helveston *et al.* 2015), marketing (Brand and Baier 2022), and health economics (van Kinschot *et al.* 2021). In each of these fields, deciders may value the same attributes differently. Some travelers pay a high fare to save time, whereas others accept a long trip if it is cheap, and such differences often stem from tastes and attitudes that are difficult to measure. This preference heterogeneity is frequently of primary interest,

because the response to a fare increase depends on how price sensitivity is distributed over the travelers, and because markets often consist of segments with distinct tastes. Empirical research therefore requires a statistical model that infers the distribution of preferences in a population of deciders from observed choices.

Statistical inference on choices among a discrete set of alternatives traditionally builds upon random utility models (Train 2009). The utility of an alternative is modeled as a linear combination of covariates plus a random error, the decider chooses the alternative with the largest utility, and only this choice is observed. The multinomial logit model has closed-form choice probabilities but implies the independence of irrelevant alternatives, whereas the multinomial probit model assumes jointly normal errors, which may be correlated across alternatives and thereby allow flexible substitution patterns. However, its choice probabilities are multivariate normal integrals without a closed form, which maximum likelihood estimation must simulate, for instance with the Geweke–Hajivassiliou–Keane simulator (Hajivassiliou *et al.* 1996), or approximate, for instance with the Solow–Joe approximation (Solow 1990; Joe 1995).

Unobserved preference heterogeneity is commonly modeled through random coefficients that take decider-specific values from a mixing distribution. This approach was introduced in the probit context by Hausman and Wise (1978) and became popular with the mixed logit model (McFadden and Train 2000) and with hierarchical Bayesian models in marketing (Allenby and Rossi 1998). The mixing distribution must be specified by the analyst and is frequently assumed to be normal (Train 2009, Chapter 6), a choice that is often limited by the software and not always checked. This assumption is violated under domain constraints, multimodal preference distributions, or asymmetric heterogeneity (Cirillo and Axhausen 2006; Scarpa *et al.* 2008; Caputo *et al.* 2018). A misspecified mixing distribution distorts derived quantities such as the value of travel time savings (Hess *et al.* 2005; Fosgerau 2006). Latent class models instead partition the deciders into segments with class-specific coefficients (Kamakura and Russell 1989; Greene and Hensher 2003; Kim and Mokhtarian 2023) and require a specification of the number of classes. For the probit model, random coefficients increase the computational burden further, because they induce dependencies across a decider's choice occasions and multiply the dimension of the likelihood integrals.

The R (R Core Team 2026) package **RprobitB**[1] (Oelschläger and Bauer 2026) approaches these challenges through Bayesian estimation, which avoids the evaluation of the likelihood integrals, admits flexible mixing distributions, and can infer the number of classes from the data. Following Albert and Chib (1993), McCulloch and Rossi (1994), and Imai and van Dyk (2005a), a Markov chain Monte Carlo (MCMC) sampler treats the latent utilities as additional parameters and draws them together with the model parameters. Conditional on the utilities, the probit model is a linear regression with normal errors, in which the coefficients have normal and the covariance matrices have inverse Wishart full conditional distributions. The decider-specific coefficients and the class allocations are also drawn from their full conditional distributions. Random coefficients follow normal or log-normal mixing distributions, the latter for coefficients whose sign is fixed by theory, and may be correlated. Combined with latent classes, they yield the latent class mixed probit model of Oelschläger and Bauer (2021), whose mixture of normals approximates skewed or multimodal preference distributions and classifies the deciders by their preferences (Oelschläger 2026a). If the number of classes is not

[1] The package name is a portmanteau of the language R, the probit model, and the Bayes framework.

Package	Probit	Bayesian	Random coefficients	Latent classes	Classes inferred
mlogit	✓	–	(✓)	–	–
gmnl	–	–	(✓)	(✓)	–
logitr	–	–	(✓)	–	–
apollo	(✓)	✓	✓	✓	–
Biogeme	(✓)	✓	✓	✓	–
bayesm	✓	✓	(✓)	(✓)	(✓)
MNP	✓	✓	–	–	–
Rchoice	(✓)	–	(✓)	–	–
Rprobit	✓	–	✓	–	–
RprobitB	✓	✓	✓	✓	✓

Table 1: Open-source software for discrete choice models with preference heterogeneity. Check marks in parentheses indicate a restriction to logit models or to binary and ordered responses.

fixed in advance, a sparse finite mixture starts with more classes than needed, and a prior that favors small class weights empties the superfluous classes during sampling (Rousseau and Mengersen 2011; Malsiner-Walli *et al.* 2016; Frühwirth-Schnatter and Malsiner-Walli 2019). Alternatively, a Dirichlet process mixture treats the number of occupied classes as random (Escobar and West 1995; Neal 2000), and the weight-based updating scheme of Oelschläger and Bauer (2021) removes, splits, and merges classes according to their weights and means. From the posterior draws, the package derives the population distribution of preferences, class membership probabilities, decider-specific coefficients, willingness-to-pay measures, marginal effects, and predictions conditional on the choice history of a decider, together with their posterior uncertainty.

The package also supports ordered and ranked responses, cross-sectional and panel data, and individual choice sets. All models are fitted with a single function `fit()`, which returns an S3 object that supports standard methods such as `summary()`, `plot()`, `coef()`, `predict()`, and `update()`. Several Gibbs chains can run in parallel through the **future** framework (Bengtsson 2021). The draws are stored in the format of the **posterior** package (Bürkner *et al.* 2026), which provides the convergence diagnostics of Vehtari *et al.* (2021), and are displayed with **bayesplot** (Gabry and Mahr 2026). Choice data handling and simulation come from the companion package **choicedata** (Oelschläger 2026b). Competing specifications are compared by information criteria and approximate leave-one-out cross-validation with the **loo** package (Vehtari *et al.* 2017, 2026), and by Bayes factors estimated by bridge sampling (Meng and Wong 1996; Gronau *et al.* 2020).

Existing open-source software covers the combination of probit models and preference heterogeneity only in part, as Table 1 summarizes. The R packages **mlogit** (Croissant 2020), **gmnl** (Sarrias and Daziano 2017), **logitr** (Helveston 2023), and **apollo** (Hess and Palma 2019), as well as **Biogeme** (Bierlaire 2020, 2025) for Python (Van Rossum and Drake 2009), model heterogeneity in logit models with a mixing distribution or a number of classes that the analyst fixes in advance, by maximum (simulated) likelihood or, in the case of **apollo** and **Biogeme**, also by Bayesian estimation. **bayesm** (Rossi 2025) relaxes this restriction through finite and Dirichlet process mixtures of normals, but only for the logit model. For the probit model, in contrast, the multinomial models of **mlogit**, **bayesm**, and **MNP** (Imai and van Dyk 2005b, 2026) have fixed coefficients, and **apollo**, **Biogeme**, and **Rchoice** (Sarrias 2016) offer random coefficients for binary and ordered responses only. The frequentist counterpart of **RprobitB**

is **Rprobit** (Bauer *et al.* 2024). It estimates multinomial probit models with normal mixing distributions by maximum likelihood or by the maximum approximate composite marginal likelihood approach of Bhat (2011), with analytic approximations of the normal probabilities such as the Solow–Joe approximation. To the best of our knowledge, **RprobitB** is the only package that estimates multinomial probit models with random coefficients and correlated errors in a Bayesian framework, and the only one that infers the number of latent classes of such a model from the data. The package is available from the Comprehensive R Archive Network (CRAN) at `https://CRAN.R-project.org/package=RprobitB`, and its development version is hosted at `https://github.com/loelschlaeger/RprobitB`.

The article proceeds as follows. Section 2 defines the probit model with random coefficients and latent classes. In Section 3, we introduce the package's estimation routine `fit()` and describe the prior distributions and the Gibbs sampler. Section 4 interprets the posterior of a fitted model. Section 5 models preference heterogeneity through random coefficients and latent classes, whose number is fixed or inferred from the data. Section 6 treats ordered and ranked responses, Sections 7 and 8 cover prediction, residuals, and model comparison, and Section 9 concludes the paper.

2. The probit model

Assume that we observe the choices of N deciders, where decider $n = 1, \dots, N$ faces $T_n \geq 1$ choice occasions. At occasion $t = 1, \dots, T_n$, the decider chooses among a finite set of $J \geq 2$ mutually exclusive and exhaustive alternatives (Train 2009, Chapter 2). The case $T_n = 1$ for all n corresponds to a cross section, and $T_n > 1$ to a panel data set, which may be unbalanced. With $J = 2$, the model is the binary probit model.

2.1. Utilities and choices

Every alternative $j = 1, \dots, J$ provides decider n at occasion t with a latent utility

$$U_{ntj} = X_{ntj}^\top \theta_n + \varepsilon_{ntj}, \tag{1}$$

where $X_{ntj} \in \mathbb{R}^P$ collects the covariates and $\theta_n \in \mathbb{R}^P$ the coefficients of decider n. The error vector $\varepsilon_{nt} = (\varepsilon_{nt1}, \dots, \varepsilon_{ntJ})^\top \sim \mathrm{N}_J(0, \Sigma)$ is multivariate normal with a $J \times J$ covariance matrix Σ and independent across deciders and occasions. The decider chooses the available alternative with the largest utility,

$$y_{nt} = \operatorname*{argmax}_{j \in \mathcal{J}_{nt}} U_{ntj}, \tag{2}$$

where $y_{nt} = j$ denotes the event that decider n chooses alternative j at occasion t, and $\mathcal{J}_{nt} \subseteq \{1, \dots, J\}$ is the set of alternatives that are available at this occasion.

2.2. Covariate types and the model formula

Covariates differ in whether their values and their coefficients vary across alternatives. The price of a product is alternative-specific and usually has the same coefficient in every utility. The income of a buyer is constant across alternatives and can only enter through alternative-specific coefficients. Like the price, the travel time varies across alternatives, but its coefficient

may differ between a car, which the traveler drives, and a train, in which the traveler can read or work. To distinguish these three types, we write (1) as

$$U_{ntj} = \theta_j^0 + A_{ntj}^\top \theta^A + B_{nt}^\top \theta_j^B + C_{ntj}^\top \theta_j^C + \varepsilon_{ntj}, \tag{3}$$

where the decider index of the coefficients is suppressed. The covariates A_{ntj} and C_{ntj} depend on the alternative, while B_{nt} depends on the decider and the occasion only. The coefficient θ^A is the same for all alternatives, whereas θ_j^B and θ_j^C are alternative-specific. The intercepts θ_j^0 are the alternative-specific constants (ASCs). **RprobitB** specifies (3) through a '`formula`' object of the form `choice ~ A | B | C`, where `choice` is the response variable and `A`, `B`, and `C` are the covariates of the three types. For example, the formula `choice ~ price | income | time` gives the price one coefficient for all alternatives, and the income and the travel time one coefficient per alternative. The notation follows **mlogit** and builds on the multi-part formulas of the **Formula** package (Zeileis and Croissant 2010). ASCs are included by default and removed by `0` in the second part, as in `choice ~ A | 0 | C`. We call every coefficient of (3) an effect. The package labels every effect by its covariate and, if the coefficient is alternative-specific, by the alternative. For the formula above and the alternatives bus, car, and train, the labels include `price`, `income_car`, `time_bus`, and `ASC_car`.

2.3. Normalization

Utilities are identified only up to level and scale. Adding the same constant to all U_{ntj} or multiplying all of them by a positive number leaves the choice behavior implied by (2) unchanged (Train 2009, Chapter 2). We normalize the level by taking differences with respect to a base alternative k,

$$\tilde{U}_{ntj} = \tilde{X}_{ntj}^\top \theta_n + \tilde{\varepsilon}_{ntj}, \qquad j \neq k, \tag{4}$$

with $\tilde{U}_{ntj} = U_{ntj} - U_{ntk}$, $\tilde{X}_{ntj} = X_{ntj} - X_{ntk}$, and $\tilde{\varepsilon}_{ntj} = \varepsilon_{ntj} - \varepsilon_{ntk}$. The vector $\tilde{\varepsilon}_{nt}$ of the $J-1$ error differences is multivariate normal with mean zero and a $(J-1)\times(J-1)$ covariance matrix $\tilde{\Sigma}$. Differencing also shows that only the differences of the ASCs and of the type `B` coefficients from those of the base are identified. The base coefficients θ_k^0 and θ_k^B are hence fixed at zero, and the remaining ones are interpreted relative to the base, which is the first alternative in alphabetical order unless the argument `base` of `fit()` names another one.

The scale is normalized by fixing one parameter, by default the error variance of the first utility difference, $\tilde{\Sigma}_{11} = 1$. The argument `scale` of `fit()` selects a different normalization, for example `scale = c(price = -1)`, which fixes a coefficient instead of a variance and measures every other coefficient in units of the fixed one. With a price coefficient of -1, every other coefficient has a direct willingness-to-pay interpretation, as in the electricity example of Section 5.1.

2.4. Random coefficients

Preferences can differ between deciders (Ben-Akiva *et al.* 1993), and the repeated choices of a decider are informative about the coefficients of this decider (Allenby and Rossi 1998). **RprobitB** therefore splits the coefficient vector into P_f fixed effects $\alpha \in \mathbb{R}^{P_f}$ that are common to all deciders and P_r random effects with a decider-specific value. The random effects are functions of a latent vector $\beta_n \in \mathbb{R}^{P_r}$ that is drawn independently for every decider from a

Code	Distribution	Correlated	Coefficient $g_p(\beta_{np})$
`"cn"`	normal	yes	β_{np}, any sign
`"n"`	normal	no	β_{np}, any sign
`"cln"`	log-normal	yes	$\exp(\beta_{np}) > 0$
`"ln"`	log-normal	no	$\exp(\beta_{np}) > 0$
`"cln-"`	negative log-normal	yes	$-\exp(\beta_{np}) < 0$
`"ln-"`	negative log-normal	no	$-\exp(\beta_{np}) < 0$

Table 2: The six mixing distributions of a random effect, their codes in the argument `random_effects`, and the coefficient of (6) that enters the utility.

multivariate normal distribution,

$$\beta_n \sim \mathrm{N}_{P_r}(b, \Omega), \qquad \theta_n = \left(\alpha^\top, g(\beta_n)^\top\right)^\top, \tag{5}$$

independently of the covariates and the errors. The mean vector b and the covariance matrix Ω of this distribution are parameters of the model. The map g acts componentwise and determines the mixing distribution of each effect $p = 1, \dots, P_r$,

$$g_p(\beta_{np}) = \begin{cases} \beta_{np} & \text{for a normal effect,} \\ \exp(\beta_{np}) & \text{for a log-normal effect,} \\ -\exp(\beta_{np}) & \text{for a negative log-normal effect.} \end{cases} \tag{6}$$

A normal effect admits coefficients of either sign, distributed as $\mathrm{N}(b_p, \Omega_{pp})$ over the population. A log-normal effect restricts the coefficient to positive values, with median $\exp(b_p)$ and mean $\exp(b_p + \Omega_{pp}/2)$ over the population. A negative log-normal effect is the negative of a log-normal variable and is appropriate when theory implies a negative sign, for example of a price coefficient. A correlated effect has free covariances in Ω with the other correlated effects, whereas an uncorrelated effect has zeros in its row and column of Ω apart from the variance on the diagonal. Table 2 lists the six resulting mixing distributions with the codes by which the argument `random_effects` of `fit()` requests them, as in `random_effects = c(price = "cln-", time = "cn")`. For log-normal effects, b, Ω, and the individual coefficients β_n are reported on the latent normal scale of (5).

2.5. Latent classes

A single normal distribution is a restrictive assumption about how preferences are distributed over a population (Train 2009, Section 6.2). **RprobitB** follows Oelschläger and Bauer (2021) and replaces the normal distribution in (5) by a mixture of C normal distributions,

$$\beta_n \sim \sum_{c=1}^{C} s_c \, \mathrm{N}_{P_r}(b_c, \Omega_c), \tag{7}$$

with class weights $s = (s_1, \dots, s_C)$ satisfying $s_c > 0$ and $\sum_c s_c = 1$, and class-specific means b_c and covariances Ω_c. The mixture is equivalent to a latent class model. An allocation variable $z_n \in \{1, \dots, C\}$ assigns decider n to class c with probability $\mathsf{P}(z_n = c) = s_c$, and conditional on the allocation, $\beta_n \sim \mathrm{N}_{P_r}(b_{z_n}, \Omega_{z_n})$. Because the allocations are parameters with a posterior distribution, the model classifies deciders by their preferences (Oelschläger

2026a). The argument `latent_class_effects` of `fit()` names the effects that differ between the latent classes. A random effect named there follows (7), whereas a random effect not named there keeps one mean and one covariance for all classes and is uncorrelated with the effects named there. A nonrandom effect named there takes one value per class and does not vary within a class. With the P_l effects of the latter kind collected in $\lambda_c \in \mathbb{R}^{P_l}$, the complete coefficient vector of decider n is

$$\theta_n = \left(\alpha^\top, \lambda_{z_n}^\top, g(\beta_n)^\top\right)^\top, \qquad P = P_f + P_l + P_r. \tag{8}$$

If $P_l > 0$ but $P_r = 0$, (8) is the classical latent class model, in which all heterogeneity is due to the class membership (Kamakura and Russell 1989; Greene and Hensher 2003). If instead $P_r > 0$, it is the latent class mixed probit model of Oelschläger and Bauer (2021). The number of classes is either fixed by the argument `classes` of `fit()`, or the number of occupied classes is inferred from the data. The argument `class_update` selects between a finite mixture whose classes all stay occupied (`"fixed"`), an overfitted finite mixture with a sparse prior on the weights that empties superfluous classes (`"sparse"`; Rousseau and Mengersen 2011; Malsiner-Walli *et al.* 2016; Frühwirth-Schnatter and Malsiner-Walli 2019), a Dirichlet process mixture whose number of occupied classes is random (`"dirichlet_process"`; Escobar and West 1995; Neal 2000), and the weight-based remove, split, and merge heuristic of Oelschläger and Bauer (2021) (`"weight_based"`). Section 3 describes the corresponding sampler steps, and Section 5.4 applies all four updating schemes to empirical and simulated data.

2.6. Ordered and ranked responses

Some responses are ordered levels, such as the agreement with a statement on a Likert scale, for which one utility per level is not natural (Train 2009, Section 7.4). Instead, the model has a single utility

$$U_{nt} = \theta_n^0 + X_{nt}^\top \theta_n + \varepsilon_{nt}, \qquad \varepsilon_{nt} \sim \mathrm{N}(0, \sigma^2). \tag{9}$$

The utility falls into one of J intervals that are linked to the ordered levels $j = 1, \ldots, J$,

$$y_{nt} = j \quad \text{if and only if} \quad \gamma_{j-1} < U_{nt} \leq \gamma_j, \tag{10}$$

with thresholds $\gamma_0 = -\infty < \gamma_1 < \cdots < \gamma_{J-1} < \gamma_J = +\infty$. The thresholds are parameterized through logarithmic increments $d \in \mathbb{R}^{J-2}$ with $\gamma_j = \sum_{i=1}^{j-1} \exp(d_i)$ for $j = 2, \ldots, J-1$. The level is normalized by $\gamma_1 = 0$ and the scale by $\sigma^2 = 1$. Random coefficients and latent classes apply to (9) exactly as to (1).

Some surveys record a complete ranking of the alternatives, which is more informative about the preferences (Train 2009, Section 7.3), as we demonstrate in Section 6.2. The utilities of (1) stay the same, and only the link changes. The response at occasion t is a permutation $r_{nt} = (r_{nt1}, \ldots, r_{ntJ})$ of the alternatives, ordered from best to worst, and the model requires

$$U_{ntr_{nt1}} > U_{ntr_{nt2}} > \cdots > U_{ntr_{ntJ}}. \tag{11}$$

2.7. Choice probabilities and likelihood

Conditional on θ_n and $\tilde{\Sigma}$, the utility differences of an unordered choice are distributed as $\mathrm{N}_{J-1}(\tilde{X}_{nt}\theta_n, \tilde{\Sigma})$, and the choice rule of (2) yields the choice probability

$$\mathsf{P}(y_{nt} = j \mid X_{nt}, \theta_n, \tilde{\Sigma}) = \begin{cases} \mathsf{P}(\tilde{U}_{ntj} > 0,\ \tilde{U}_{ntj} > \tilde{U}_{ntl} \text{ for all } l \notin \{j, k\}) & \text{if } j \neq k, \\ \mathsf{P}(\tilde{U}_{ntl} < 0 \text{ for all } l \neq k) & \text{if } j = k, \end{cases} \tag{12}$$

where k is the base alternative of Section 2.3. With individual choice sets, the inequalities run over the available alternatives only. For a ranked response, the probability of the ranking r_{nt} is the probability that all $J-1$ utility differences between consecutively ranked alternatives in (11) are positive. For an ordered response, the probability of level j follows from (10),

$$\mathsf{P}(y_{nt} = j \mid X_{nt}, \theta_n, \gamma, \sigma^2) = \Phi\Big(\frac{\gamma_j - \theta_n^0 - X_{nt}^\top \theta_n}{\sigma}\Big) - \Phi\Big(\frac{\gamma_{j-1} - \theta_n^0 - X_{nt}^\top \theta_n}{\sigma}\Big), \tag{13}$$

with the standard normal distribution function Φ.

The population parameters of a model are those without a decider index: the fixed coefficients α, the class coefficients λ_c, the means b_c and covariances Ω_c of the random coefficients, the class weights s, the error covariance $\tilde{\Sigma}$ or the error variance σ^2, and the thresholds γ. We collect them in the vector ϑ. The random coefficients β_n and the allocations z_n are decider-level parameters, and the likelihood contribution of a decider integrates them out. Let $\mathsf{P}(y_{nt} \mid X_{nt}, \theta_n, \tilde{\Sigma})$ denote the probability of the observed response y_{nt}, which is (12), its ranked analog, or (13) with γ and σ^2 in place of $\tilde{\Sigma}$. With $y_n = (y_{n1}, \dots, y_{nT_n})$ and X_n the responses and covariates of decider n, the likelihood of the decider is

$$p(y_n \mid X_n, \vartheta) = \sum_{c=1}^{C} s_c \int \prod_{t=1}^{T_n} \mathsf{P}(y_{nt} \mid X_{nt}, \theta_n, \tilde{\Sigma})\, \phi(\beta_n \mid b_c, \Omega_c)\, \mathrm{d}\beta_n, \tag{14}$$

where $\theta_n = (\alpha^\top, \lambda_c^\top, g(\beta_n)^\top)^\top$ is the coefficient vector of (8) for a member of class c, and $\phi(\cdot \mid b_c, \Omega_c)$ is the density of $\mathrm{N}_{P_r}(b_c, \Omega_c)$. The likelihood of the sample $y = (y_1, \dots, y_N)$ is the product of (14) over all deciders, since the deciders are assumed to be independent given the parameters.

3. Bayesian estimation with `fit()`

RprobitB estimates every model of Section 2 with a Gibbs sampler. The sampler builds on McCulloch and Rossi (1994) and Imai and van Dyk (2005a) for the multinomial probit model, on Albert and Chib (1993) for the ordered probit model, and on Oelschläger and Bauer (2021) for the latent class mixed probit model. It relies on data augmentation (Albert and Chib 1993), which treats the latent utilities as additional parameters. Conditional on the parameters, the utilities follow truncated normal distributions whose truncation points are determined by the observed responses. The sampler is written in C++ for fast computation and linked to R through **Rcpp** (Eddelbuettel and François 2011) and **RcppArmadillo** (Eddelbuettel and Sanderson 2014). In this section, we introduce `fit()`, describe the prior distributions and the sampler, and fit models to simulated data and to the choices of train travelers in a stated choice experiment.

3.1. The function `fit()`

The function `fit()` takes a formula and a data frame, runs the Gibbs sampler, and returns an object of class '`RprobitB_fit`', on which the methods presented in the following sections operate. Table 3 groups its arguments by their purpose. The help page of `fit()`, accessible via `help(fit, package = "RprobitB")`, documents each of them.

`fit()` accepts a '`data.frame`' in the wide or in the long layout, selected by the argument `format`. In the wide layout, every row is one choice occasion, and a covariate that varies across

Purpose	Arguments of `fit()`	Section
Model	`formula`, `choice_type`, `alternatives`, `base`, `scale`	2.2, 2.3, 2.6
Heterogeneity	`random_effects`, `latent_class_effects`, `classes`, `class_update`, `max_classes`, `weight_based_control`	2.4, 2.5, 5
Data	`data`, `format`, `column_decider`, `column_occasion`, `column_alternative`, `delimiter`	3.1
Prior	`prior`	3.2
Sampler	`iterations`, `warmup`, `thin`, `chains`, `save_individual_draws`, `progress`	3.3, 5.2
Simulation	`n_deciders`, `n_occasions`, `n_alternatives`, `covariates`, `dgp_parameters`	3.4

Table 3: Arguments of `fit()`, grouped by purpose, and the related sections.

alternatives occupies one column per alternative, named by the covariate, the `delimiter`, and the alternative, such as `price_bus` and `price_car`. In the long layout, every row relates to one alternative of one choice occasion, and the argument `column_alternative` names the column that identifies the alternative. An occasion may then comprise only the rows of its available alternatives, which specifies the individual choice sets of (2). The arguments `column_decider` and `column_occasion` name the columns that identify the decider and, in panel data, also the choice occasion.

By default, the sampler runs four chains of 1000 iterations each, half of them warmup. With `data = NULL`, `fit()` first simulates a data set from the specified model and then fits it (Section 3.4). The returned object holds, among other components, the data as a '`choice_data`' object of **choicedata**, the hyperparameters that were used, and the retained draws of all chains as a '`draws_array`' object of **posterior**.

3.2. Prior distributions

The priors are weakly informative by default and conjugate, which gives the normal, inverse Wishart, and Dirichlet full conditional distributions of Section 3.3. Table 4 lists them with their default hyperparameters and the names under which the argument `prior` of `fit()` accepts replacements. The coefficients α and λ_c and the class means b_c have normal priors centered at zero with variance 10. The covariance matrices Ω_c of the random coefficients and $\tilde{\Sigma}^*$ of the errors have inverse Wishart priors with ν and κ degrees of freedom and scale matrices Θ and Λ. As in Imai and van Dyk (2005a), the prior of the error covariance is placed on the matrix $\tilde{\Sigma}^*$ before the scale normalization, with $\tilde{\Sigma} = \tilde{\Sigma}^*/\tilde{\Sigma}^*_{11}$ under the default normalization. The priors of λ_c, b_c, and Ω_c are independent over the classes. The class weights s of a finite mixture have a symmetric Dirichlet prior with concentration parameter δ, which by default is fixed at one for `class_update = "fixed"` and `"weight_based"`. For the sparse finite mixture, δ receives a gamma hyperprior with shape a_δ and rate b_δ, by default $(a_\delta, b_\delta) = (1, 200)$ as in Frühwirth-Schnatter and Malsiner-Walli (2019). Under the Dirichlet process, δ is the concentration parameter of the process, with the hyperprior $(a_\delta, b_\delta) = (2, 4)$ of Escobar and West (1995). The logarithmic threshold increments d of an ordered model have a standard normal prior.

Parameter	Prior	Default	Components of `prior`
α	$\mathrm{N}_{P_f}(\psi, \Psi)$	$\psi = 0$, $\Psi = 10 I_{P_f}$	`fixed_mean` `fixed_covariance`
λ_c	$\mathrm{N}_{P_l}(\psi_\lambda, \Psi_\lambda)$	$\psi_\lambda = 0$, $\Psi_\lambda = 10 I_{P_l}$	`latent_class_mean` `latent_class_covariance`
b_c	$\mathrm{N}_{P_r}(\xi, \Xi)$	$\xi = 0$, $\Xi = 10 I_{P_r}$	`random_mean` `random_mean_covariance`
Ω_c	$\mathrm{W}^{-1}_{P_r}(\nu, \Theta)$	$\nu = P_r + 2$, $\Theta = I_{P_r}$	`random_covariance_df` `random_covariance_scale`
s	$\mathrm{Dir}_C(\delta, \ldots, \delta)$	$\delta = 1$	`class_concentration`
δ	$\mathrm{Gamma}(a_\delta, b_\delta)$	$(a_\delta, b_\delta) = (1, 200)$ or $(2, 4)$	`class_concentration`
$\tilde{\Sigma}^*$	$\mathrm{W}^{-1}_{J-1}(\kappa, \Lambda)$	$\kappa = J + 1$, $\Lambda = I_{J-1}$	`error_covariance_df` `error_covariance_scale`
d	$\mathrm{N}_{J-2}(\zeta, \Upsilon)$	$\zeta = 0$, $\Upsilon = I_{J-2}$	`threshold_mean` `threshold_covariance`

Table 4: Prior distributions, their default hyperparameters, and the corresponding components of the argument `prior` of `fit()`. The Dirichlet prior of s applies to finite mixtures; under the Dirichlet process, δ is the concentration parameter of the process.

3.3. The Gibbs sampler

For unordered and ranked responses, the latent variables that augment the data are the utility differences $\tilde{U}_{nt} \in \mathbb{R}^{J-1}$ of (4). Their systematic part is

$$\mu_{nt} = \tilde{W}_{nt}\alpha + \tilde{L}_{nt}\lambda_{z_n} + \tilde{V}_{nt}\, g(\beta_n), \tag{15}$$

where $\tilde{W}_{nt}$, $\tilde{L}_{nt}$, and $\tilde{V}_{nt}$ are the columns of the differenced covariates that belong to α, λ_c, and β_n, and the error of $\tilde{U}_{nt}$ is $\tilde{\varepsilon}_{nt} \sim \mathrm{N}_{J-1}(0, \tilde{\Sigma})$. Given the utilities and $\tilde{\Sigma}$, the model is a linear regression. Every coefficient block $\rho \in \mathbb{R}^{P_\rho}$ with prior $\mathrm{N}_{P_\rho}(\rho_0, V_0)$ and design matrices Z_{nt} hence has the normal full conditional distribution

$$\begin{aligned} &\rho \mid \cdot \sim \mathrm{N}_{P_\rho}(\Sigma_\rho\, \mu_\rho, \Sigma_\rho), \\ &\mu_\rho = V_0^{-1}\rho_0 + \sum_{n,t} Z_{nt}^\top \tilde{\Sigma}^{-1}(\tilde{U}_{nt} - q_{nt}), \qquad \Sigma_\rho = \Big(V_0^{-1} + \sum_{n,t} Z_{nt}^\top \tilde{\Sigma}^{-1} Z_{nt}\Big)^{-1}, \end{aligned} \tag{16}$$

where q_{nt} is the part of μ_{nt} that does not depend on ρ. Every chain starts at a draw from the prior distributions of the coefficients and thresholds, with the random coefficients at their class means, Ω_c at the prior scale Θ, $\tilde{\Sigma}$ at the identity, the deciders allocated to the classes at random, and the utilities at zero, so that the chains start dispersed. One iteration then updates the following blocks in turn:

1. The class weights s of a finite mixture are drawn from $\mathrm{Dir}_C(\delta + m_1, \ldots, \delta + m_C)$, where m_c is the number of deciders in class c. Under the Dirichlet process, the weights are integrated out by the algorithm of Neal (2000), and the package reports the class shares m_c/N.

2. For every class c, the mean b_c and the covariance Ω_c follow the conjugate normal and inverse Wishart updates given the sample $\{\beta_n : z_n = c\}$.

3. The class coefficients λ_c follow (16) with $Z_{nt} = \tilde{L}_{nt}$ and the sums over the deciders of class c.

4. For every decider n, the random coefficients β_n follow (16) with $Z_{nt} = \tilde{V}_{nt}$, the sums over the occasions of decider n, and the class distribution $\mathrm{N}_{P_r}(b_{z_n}, \Omega_{z_n})$ as the prior. With a log-normal random effect, an elliptical slice sampling step (Murray *et al.* 2010) replaces this update. The fixed coefficients α then follow (16) with $Z_{nt} = \tilde{W}_{nt}$ and the sums over all deciders.

5. Decider n is allocated to class c with probability

$$\mathsf{P}(z_n = c \mid \cdot) \propto s_c \, \phi(\beta_n \mid b_c, \Omega_c) \prod_{t=1}^{T_n} \mathsf{P}(y_{nt} \mid X_{nt}, \alpha, \lambda_c, \beta_n, \tilde{\Sigma}). \tag{17}$$

   The four values of `class_update` differ in the treatment of the number of classes. For `"fixed"`, the last member of a class stays in it, which keeps all C classes occupied. For `"sparse"`, C is an upper bound, and the small concentration parameter empties the classes that the data do not require. For `"dirichlet_process"`, deciders open new classes, up to `max_classes`, by Algorithm 8 of Neal (2000). For `"weight_based"`, the heuristic of Oelschläger and Bauer (2021) removes, splits, or merges classes during the warmup.

6. In an ordered model, the thresholds γ are updated by a random-walk Metropolis–Hastings step on their logarithmic increments d.

7. Given the parameters, the utility differences $\tilde{U}_{nt}$ of an unordered choice are $\mathrm{N}_{J-1}(\mu_{nt}, \tilde{\Sigma})$ truncated to the region in which the chosen alternative has the largest utility, and the truncated normal draws use the samplers of Robert (1995). For a ranked response, the truncation is chosen such that the observed ranking holds. An ordered response has a single utility $U_{nt} = \mu_{nt} + \varepsilon_{nt}$ with $\varepsilon_{nt} \sim \mathrm{N}(0, 1)$, drawn from $\mathrm{N}(\mu_{nt}, 1)$ truncated to $(\gamma_{y_{nt}-1}, \gamma_{y_{nt}}]$.

8. The error covariance is drawn from $\tilde{\Sigma}^* \sim \mathrm{W}^{-1}_{J-1}(\kappa + \sum_n T_n, \Lambda + S)$ with the scatter matrix S of the residuals $\tilde{U}_{nt} - \mu_{nt}$ and normalized to $\tilde{\Sigma} = \tilde{\Sigma}^* / \tilde{\Sigma}^*_{11}$.

Of the `iterations` of every chain, the first `warmup` are discarded and every `thin`-th of the remaining ones is retained. The D retained draws of all chains are rescaled to the normalization requested by `scale`. In a mixture model, a chain may swap the class labels during a run (Stephens 2000). The draws are therefore relabeled after sampling by matching them to a reference partition (Dahl 2006; Papastamoulis and Iliopoulos 2010), and the classes are numbered by decreasing posterior mean weight.

3.4. Parameter recovery with simulated data

With `data = NULL`, `fit()` simulates the choices from the specified model before it fits the model, which can be used to verify that the sampler recovers the parameters of a known data generating process (DGP). As a demonstration example below, we simulate 400 deciders who choose 20 times among bus, car, and train and who belong to one of three latent classes. The formula contains the three covariate types of Section 2.2: the price with one coefficient for

all alternatives, which we specify as a normal random effect whose mean and variance differ between the classes, the income of the decider with one coefficient per alternative, and the travel time, whose values and coefficients both differ between the alternatives. We collect the parameters of the DGP in a list containing, for each class, the coefficients `beta`, whose entry `price` is the mean of the random price coefficient, and the variance `Omega` of this coefficient, as well as the class `weights` and the covariance matrix `Sigma` of the three errors. Covariates that are not supplied through the argument `covariates` are drawn from a standard normal distribution.

```
R> common <- c(
+    income_car = 0.5, income_train = -0.5, ASC_car = 1, ASC_train = 0.5,
+    time_bus = -0.5, time_car = -1, time_train = -1.5
+  )
R> dgp_parameters <- list(
+    beta = list(
+      c(price = -1.75, common), c(price = 0, common),
+      c(price = 1.75, common)
+    ),
+    Omega = list(matrix(0.05), matrix(0.05), matrix(0.05)),
+    weights = c(1, 1, 1) / 3, Sigma = diag(c(0.5, 0.5, 1))
+  )
```

`fit()` simulates the data from these parameters and fits the model:

```
R> simulation <- fit(
+    choice ~ price | income | time, random_effects = "price",
+    latent_class_effects = "price", classes = 3,
+    alternatives = c("bus", "car", "train"), base = "bus",
+    n_deciders = 400, n_occasions = 20, dgp_parameters = dgp_parameters,
+    iterations = 40000, warmup = 20000, thin = 20, chains = 2
+  )
```

`model.frame()` returns the simulated data in the wide layout:

```
R> head(model.frame(simulation), 3)
```

```
  deciderID occasionID choice     income  price_bus   price_car
1         1          1  train  0.4874291 -0.6264538  0.18364332
2         1          2  train -2.2146999  0.7383247  0.57578135
3         1          3    car  0.9189774  1.1249309 -0.04493361
  price_train  time_bus  time_car time_train
1 -0.83562861 1.5952808 0.3295078 -0.8204684
2 -0.30538839 1.5117812 0.3898432 -0.6212406
3 -0.01619026 0.9438362 0.8212212  0.5939013
```

`summary()` lists the parameters of the DGP in the column `dgp`, in the normalization of the estimates. The entries of `Sigma` therefore refer to the utility differences with respect to bus, the base alternative, with the variance of the first difference fixed at one (Section 2.3):

```
R> summary(simulation)

Bayesian probit choice model
Formula: choice ~ price | income | time
Samples: 1000 retained per chain, 2 chains

             variable    dgp    mean    mode     sd rhat ess_bulk
     beta[income_car]  0.500  0.5260  0.5320 0.0301 1.00     1019
   beta[income_train] -0.500 -0.4847 -0.4798 0.0372 1.00     1103
        beta[ASC_car]  1.000  0.9888  0.9913 0.0338 1.00      767
      beta[ASC_train]  0.500  0.4884  0.4962 0.0353 1.00     1034
       beta[time_bus] -0.500 -0.4793 -0.4773 0.0265 1.00     1019
       beta[time_car] -1.000 -1.0346 -1.0265 0.0347 1.00      650
     beta[time_train] -1.500 -1.5614 -1.5717 0.0514 1.00      478
            weight[1]  0.333  0.3589  0.3618 0.0246 1.00     1636
            weight[2]  0.333  0.3274  0.3239 0.0236 1.00     1633
            weight[3]  0.333  0.3138  0.3122 0.0244 1.00     1418
          mu[price,1] -1.750 -1.8288 -1.8287 0.0631 1.00      618
          mu[price,2]  1.750  1.8779  1.8779 0.0636 1.00      559
          mu[price,3]  0.000  0.0102  0.0105 0.0391 1.01     1062
 Omega[price,price,1]  0.050  0.1262  0.1172 0.0374 1.00      609
 Omega[price,price,2]  0.050  0.1078  0.0927 0.0350 1.00      616
 Omega[price,price,3]  0.050  0.0798  0.0700 0.0214 1.00      916
     Sigma[car,train]  0.500  0.4647  0.4613 0.0666 1.01      570
   Sigma[train,train]  1.500  1.5837  1.5895 0.1505 1.00      735
```

`summary()` also assesses the draws with the diagnostics of Vehtari *et al.* (2021). The column `rhat` is the potential scale reduction factor $\hat{R}$, which compares the variation between and within the halves of all chains. It exceeds one if the half-chains differ in location or spread, which indicates that the chains have not converged, and should be at most 1.01. The column `ess_bulk` is the bulk effective sample size, the number of independent draws that estimate the center of the posterior as precisely as the retained draws, and should be at least 100 per chain. Both diagnostics are within these bounds, and the posterior means are close to the parameters of the DGP.

3.5. A stated choice experiment on train trips

In 1987, a survey commissioned by the Dutch national railways presented travelers with pairs of hypothetical train trips and asked which one they would book (Ben-Akiva *et al.* 1993). The two trips of a pair can differ in price, travel time, number of changes, and comfort class. The `Train` data are provided by the **mlogit** package. The prices are recorded in cents of Dutch guilders and the travel times in minutes. We convert the prices to euro at the fixed conversion rate of 2.20371 guilders per euro and the times to hours.

```
R> data("Train", package = "mlogit")
R> Train$price_A <- Train$price_A / 100 / 2.20371
R> Train$price_B <- Train$price_B / 100 / 2.20371
```

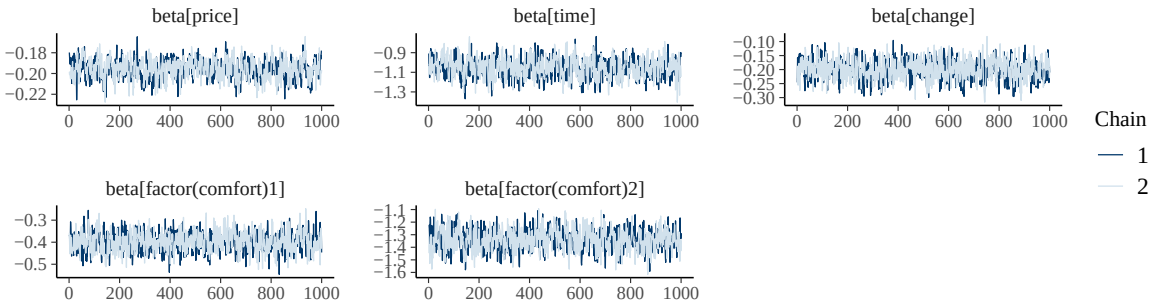


Figure 1: Trace plots of the two chains of `model_train`.

```
R> Train$time_A <- Train$time_A / 60
R> Train$time_B <- Train$time_B / 60
```

The data contain 2929 choices of 235 travelers, between 5 and 19 per traveler. The comfort class is coded as a number from `0`, the best class, to `2`, the worst, and `factor(comfort)` converts it into two dummy variables relative to the best class.

```
R> model_train <- fit(
+    choice ~ price + time + change + factor(comfort) | 0,
+    data = Train, column_decider = "id", column_occasion = "choiceid",
+    iterations = 2000, warmup = 1000, chains = 2
+  )
R> summary(model_train)

Bayesian probit choice model
Formula: choice ~ price + time + change + factor(comfort) | 0 | 0
Samples: 1000 retained per chain, 2 chains

                variable   mean   mode      sd rhat ess_bulk
            beta[price] -0.196 -0.195 0.00919    1      430
             beta[time] -1.054 -1.056 0.09730    1      557
           beta[change] -0.200 -0.190 0.03526    1      722
 beta[factor(comfort)1] -0.399 -0.397 0.04349    1      769
 beta[factor(comfort)2] -1.339 -1.340 0.08346    1      636
```

The `plot()` method draws trace, rank, autocorrelation, density, interval, or pairs plots of the retained draws with **bayesplot**. Figure 1 shows the trace plots of `model_train`, in which the chains fluctuate around a constant level and occupy the same range.

```
R> plot(model_train, type = "trace")
```

4. Interpreting the posterior

We summarize the posterior of the train model and derive compensations and marginal effects. `coef()` returns the posterior mean of every population parameter as a point estimate:

```
R> coef(model_train)

           beta[price]             beta[time]           beta[change]
            -0.1961051             -1.0542737             -0.1998090
beta[factor(comfort)1] beta[factor(comfort)2]
            -0.3989673             -1.3394756
```

`confint()` returns equal-tailed credible intervals, whose probability is set by `level`:

```
R> confint(model_train, level = 0.9)

                              5%        95%
beta[price]            -0.2120047 -0.1815502
beta[time]             -1.2093794 -0.8973283
beta[change]           -0.2603227 -0.1412878
beta[factor(comfort)1] -0.4692625 -0.3260741
beta[factor(comfort)2] -1.4747306 -1.2001112
```

4.1. Compensations

The values of the coefficients depend on the scale normalization of Section 2.3, whereas their ratios are invariant to it. Because the utility is linear in the covariates, an increase of one unit in the covariate of effect p leaves the utility unchanged if the covariate of effect q changes by

$$\tau_{pq} = -\frac{\theta_p}{\theta_q} \tag{18}$$

units (Train 2009), the compensation of effect p in units of effect q. With the price as reference effect q, τ_{pq} is the willingness to pay for one unit of effect p: the price increase that leaves the utility of the alternative unchanged, which is negative for an undesired effect. `interpret()` evaluates (18) under every draw and summarizes the posterior of each ratio. For `model_train`, the compensations with the price as reference are in euro at 1987 prices:

```
R> train_compensation <- interpret(model_train, reference = "price")
R> train_compensation

1 ‘time‘ compensates -5.38 ‘price‘ (95% interval -6.26 to -4.53)
1 ‘change‘ compensates -1.02 ‘price‘ (95% interval -1.36 to -0.681)
1 ‘factor(comfort)1‘ compensates -2.04 ‘price‘ (95% interval -2.45 to -1.62)
1 ‘factor(comfort)2‘ compensates -6.84 ‘price‘ (95% interval -7.61 to -6.06)
```

A traveler is willing to pay about 5.4 euro to save one hour of travel time (95% credible interval from 4.5 to 6.3 euro), about 1.0 euro to avoid a change of trains, and about 6.8 euro for the best instead of the worst comfort class.

4.2. Marginal effects

The choice probabilities depend on the utilities through the nonlinear probit link. The partial derivative of a probability with respect to a covariate, its marginal effect, therefore varies with

the covariate values. `interpret()` computes marginal effects by numerical differentiation. Let $\pi_j(X_{nt}, \vartheta) = \mathsf{P}(y_{nt} = j \mid X_{nt}, \vartheta)$ denote the probability of alternative j at occasion t of decider n, and let X_{nt}^{+h} and X_{nt}^{-h} be X_{nt} with a numeric covariate x of alternative j shifted by $+h$ and $-h$. The marginal effect of x on π_j under draw d is the central difference

$$\mathrm{ME}_{ntj}^{(d)}(x) = \frac{\pi_j\big(X_{nt}^{+h}, \vartheta^{(d)}\big) - \pi_j\big(X_{nt}^{-h}, \vartheta^{(d)}\big)}{2h}, \tag{19}$$

where h is a small step size; effects of x on the probabilities of the other alternatives are not computed. The marginal effect at the average (MEA, `type = "mea"`) evaluates (19) at one artificial occasion whose covariates are the averages or, for factors, the modes of the observed ones, and the average marginal effect (AME, `type = "ame"`) averages it over all observed choice occasions:

```
R> train_mea <- interpret(model_train, type = "mea")
R> print(as.data.frame(train_mea), digits = 3, row.names = FALSE)
```

```
 covariate alternative     at    mean      sd   lower   upper
     price           A 15.283 -0.0782 0.00367 -0.0854 -0.0716
     price           B 15.280 -0.0782 0.00367 -0.0854 -0.0716
      time           A  2.125 -0.4206 0.03882 -0.4952 -0.3452
      time           B  2.119 -0.4206 0.03882 -0.4952 -0.3452
    change           A  0.664 -0.0797 0.01407 -0.1077 -0.0526
    change           B  0.681 -0.0797 0.01407 -0.1077 -0.0526
```

At the average occasion, where the two trips are nearly identical in terms of their attributes, one euro more on a trip lowers the probability that the trip is chosen by about 8 percentage points, one hour more by about 42 points, and one more change of trains by about 8 points. The argument `at` replaces selected averages by chosen values. With `at = c(price_A = 20, price_B = 10)`, trip `B` is chosen with a probability close to one, and one euro then changes the probability by only about 1.1 percentage points. Figure 2 illustrates this nonlinearity. `interpret()` with `type = "ame"` returns the AMEs:

```
R> train_ame <- interpret(model_train, type = "ame")
R> print(as.data.frame(train_ame), digits = 3, row.names = FALSE)
```

```
 covariate alternative    mean      sd   lower   upper
     price           A -0.0649 0.00229 -0.0693 -0.0606
     price           B -0.0649 0.00229 -0.0693 -0.0606
      time           A -0.3490 0.03026 -0.4064 -0.2887
      time           B -0.3490 0.03026 -0.4064 -0.2887
    change           A -0.0661 0.01150 -0.0892 -0.0440
    change           B -0.0661 0.01150 -0.0892 -0.0440
```

Averaged over the observed pairs of trips, one euro more lowers the probability of a trip by about 6.5 percentage points and one hour more by about 35 points.

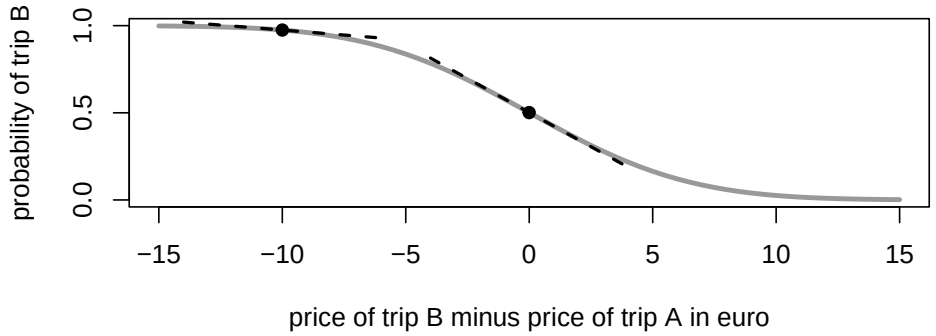


Figure 2: Posterior predictive probability of trip `B` as a function of the price difference, with all other covariates at the values of the average occasion. The points mark the average occasion and the occasion at which trip `B` costs 10 euro and trip `A` 20 euro, and the dashed lines have the slopes of the posterior mean marginal effects of the price at these occasions.

5. Modeling preference heterogeneity

We let the coefficients vary between deciders through random coefficients and latent classes. Both benefit from panel data, whose repeated choices separate this variation from the errors.

5.1. Random coefficients

The `Electricity` data of **mlogit** come from a stated choice experiment in which 361 households in the United States chose 8 to 12 times among four hypothetical suppliers (Huber and Train 2001). The suppliers differ in the price `pf` in cents per kilowatt hour (kWh), the contract length `cl` in years, whether the supplier is local (`loc`) or well-known (`wk`), and whether it offers time-of-day (`tod`) or seasonal (`seas`) rates. We insert an underscore into the column names and then number the choice occasions of every household:

```
R> data("Electricity", package = "mlogit")
R> names(Electricity) <- sub("([1-4])$", "_\\1", names(Electricity))
R> Electricity$occasion <- with(Electricity, ave(id, id, FUN = seq_along))
```

Contract length and locality receive correlated normal random coefficients. Fixing the price coefficient at -1 expresses every other coefficient in cents per kWh, that is, as a willingness to pay. We save the individual draws, from which Section 5.2 derives the coefficients of single households.

```
R> electricity <- fit(
+    choice ~ pf + cl + loc + wk + tod + seas | 0, data = Electricity,
+    random_effects = c("cl", "loc"), column_decider = "id",
+    column_occasion = "occasion", scale = c(pf = -1),
+    iterations = 25000, warmup = 10000, thin = 20, chains = 2,
+    save_individual_draws = TRUE
+  )
R> summary(electricity)

Bayesian probit choice model
Formula: choice ~ pf + cl + loc + wk + tod + seas | 0 | 0
```

```
Samples: 750 retained per chain, 2 chains

       variable    mean    mode     sd rhat ess_bulk
       beta[wk]  1.5264  1.5177 0.0720 1.00     1337
      beta[tod] -8.7309 -8.7236 0.0694 1.00     1221
     beta[seas] -9.3002 -9.3045 0.0839 1.00     1294
         mu[cl] -0.2204 -0.2228 0.0266 1.00     1401
        mu[loc]  2.0494  2.0328 0.1198 1.00     1461
   Omega[cl,cl]  0.1912  0.1881 0.0229 1.00     1156
  Omega[cl,loc]  0.0459  0.0457 0.0528 1.00     1287
 Omega[loc,loc]  2.2562  2.1318 0.3310 1.00     1332
     Sigma[2,2]  7.0711  6.9067 0.6740 1.00      460
     Sigma[2,3]  2.7906  2.7529 0.4670 1.01      445
     Sigma[3,3]  5.7520  5.6275 0.6691 1.00      806
     Sigma[2,4]  3.2917  3.2655 0.5014 1.00      394
     Sigma[3,4]  2.7051  2.6214 0.4795 1.01      476
     Sigma[4,4]  6.5200  6.3714 0.7197 1.00      681
```

The signs of `mu[cl]`, `mu[loc]`, and `beta[wk]` show that households on average dislike long contracts and prefer local and well-known suppliers. A household at the population mean accepts a price about 2.0 cents per kWh higher for a local supplier and about 1.5 cents higher for a well-known supplier, whereas each additional year of contract length has to be compensated by a price about 0.22 cents lower. The large variance `Omega[loc,loc]` shows that households differ considerably in their valuation of a local supplier.

5.2. Individual deciders

A mixed model fitted with `save_individual_draws = TRUE` retains the posterior draws of every decider's coefficients. Conditional on the population parameters, the posterior of the random coefficient vector β_n of decider n combines the population distribution of (5) with the decider's own choices,

$$p(\beta_n \mid y, \vartheta) \propto \phi(\beta_n \mid b, \Omega) \prod_{t=1}^{T_n} \mathsf{P}(y_{nt} \mid X_{nt}, \theta_n, \tilde{\Sigma}). \tag{20}$$

Equation (20) is the conditional distribution of the coefficients given the decider's choices (Train 2009, Chapter 11), in contrast to the unconditional population distribution of (5). `coef(level = "individual")` returns the posterior means of the individual coefficients, one row per household:

```
R> household_coefficients <- coef(electricity, level = "individual")
R> head(household_coefficients, 5)

           cl       loc
1  0.18463636 3.0516888
2 -0.03237544 1.6377240
3 -0.29899944 0.6366891
4 -0.54471495 1.2204165
5 -0.05685993 1.0850600
```

Because the price coefficient is fixed at -1, the `loc` column holds each household's willingness to pay for a local supplier in cents per kWh. Its first and ninth deciles show how widely the posterior mean valuations spread around the population mean `mu[loc]` of 2.05 cents:

```
R> quantile(household_coefficients[, "loc"], probs = c(0.1, 0.9))

      10%       90%
0.6887903 3.7199470
```

A tenth of the households thus have a posterior mean valuation below 0.69 cents, and another tenth above 3.72 cents.

5.3. Mixing distributions

The mixing distribution determines the support and the shape of the preference heterogeneity in the population. A normal distribution admits coefficients of both signs, whereas economic theory may restrict a coefficient, such as that of the price, to one sign, which a log-normal distribution enforces. We illustrate this with the train data of Section 3.5 and make the price coefficient a random effect, first with a normal and then with a negative log-normal mixing distribution. `update()` refits `model_train` with the named arguments replaced.

```
R> price_normal <- update(
+    model_train, random_effects = c(price = "n"),
+    iterations = 6000, warmup = 3000, thin = 30
+  )

R> price_lognormal <- update(
+    model_train, random_effects = c(price = "ln-"),
+    iterations = 6000, warmup = 3000, thin = 30
+  )
```

With a normal mixing distribution, the population density places 10 percent of its mass on positive price coefficients, whereas the negative log-normal mixing distribution excludes them. Figure 3 shows the two population densities of the price coefficient, the densities of (5) averaged over the retained draws of the population parameters. The `plot()` method draws them with `type = "mixture"`. `interpret()` evaluates the compensation of a random coefficient at the median of its mixing distribution. We compare the compensation of an hour of travel time, with its 95% credible interval, across the three models:

```
R> models <- mget(c("model_train", "price_normal", "price_lognormal"))
R> times <- lapply(models, interpret, reference = "price", effects = "time")
R> value_of_time <- sapply(times, function(x) unlist(as.data.frame(x)[-1]))
R> print(value_of_time, digits = 3)

      model_train price_normal price_lognormal
mean        -5.38       -4.573           -5.50
sd           0.44        0.352            1.03
lower       -6.26       -5.294           -7.26
upper       -4.53       -3.941           -3.22
```

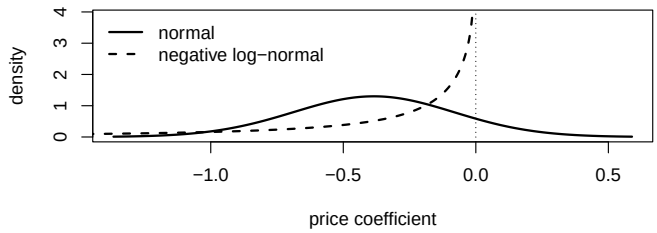


Figure 3: Population density of the price coefficient under the normal mixing distribution `"n"` (solid) and the negative log-normal mixing distribution `"ln-"` (dashed).

The value of time of the traveler with the median price coefficient is about 4.6 euro under the normal and about 5.5 euro under the negative log-normal mixing distribution, compared with 5.4 euro for the fixed coefficient of `model_train`. The choice of the mixing distribution thus shifts the estimated value of time of this traveler by almost one euro. In Section 8, we compare the three models by their predictive accuracy.

5.4. Latent classes

In the model of Section 5.1, the valuation of a local supplier follows a single normal distribution, which cannot represent skewed or multimodal valuations. To examine whether the households form segments with distinct valuations, we let this valuation follow a mixture of two normal distributions with class-specific weights, means, and variances:

```
R> electricity_classes <- update(
+    electricity, latent_class_effects = "loc", classes = 2
+  )
R> summary(electricity_classes)

Bayesian probit choice model
Formula: choice ~ pf + cl + loc + wk + tod + seas | 0 | 0
Samples: 750 retained per chain, 2 chains

         variable   mean   mode     sd rhat ess_bulk
          beta[wk]  1.525  1.508 0.0716 1.00     1414
         beta[tod] -8.737 -8.738 0.0693 1.00     1333
        beta[seas] -9.301 -9.305 0.0807 1.00     1180
         weight[1]  0.822  0.851 0.0609 1.00      357
         weight[2]  0.178  0.149 0.0609 1.00      357
         mu[loc,1]  1.568  1.556 0.1352 1.00      593
         mu[loc,2]  4.891  4.775 0.7430 1.00      408
            mu[cl] -0.222 -0.228 0.0277 1.00     1439
 Omega[loc,loc,1]  0.685  0.630 0.2088 1.00      648
 Omega[loc,loc,2]  2.295  1.373 1.3540 1.00      557
     Omega[cl,cl]  0.192  0.188 0.0236 1.00     1184
       Sigma[2,2]  7.107  7.157 0.6908 1.00      525
       Sigma[2,3]  2.766  2.697 0.4844 1.00      382
```

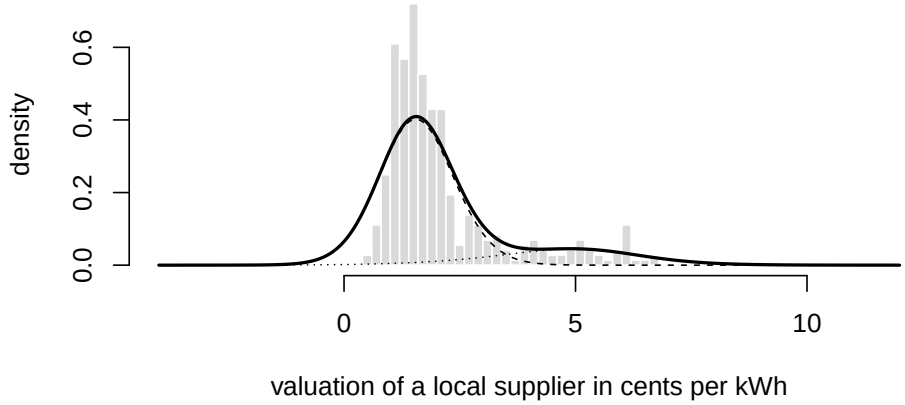


Figure 4: Mixing distribution of the valuation of a local supplier in the two-class model `electricity_classes` (solid) and its weighted class components (dashed for class 1, dotted for class 2). The histogram shows the posterior mean valuations $\beta_{n,\mathrm{loc}}$ of the 361 households under this model.

```
Sigma[3,3]  5.770  5.724 0.6677 1.00      576
Sigma[2,4]  3.277  3.193 0.5333 1.01      306
Sigma[3,4]  2.613  2.631 0.4789 1.00      426
Sigma[4,4]  6.404  6.207 0.7514 1.01      400
```

The larger class, 82 percent of the households, values a local supplier at about 1.6 cents per kWh, the smaller class at about 4.9 cents. Figure 4 shows the mixing distribution of the valuation under this model, the mixture of the two class-specific normal distributions averaged over the retained draws, together with the posterior mean valuations of the households.

A Dirichlet process mixture makes the number of occupied classes part of the posterior. Given its concentration δ, the prior expected number of occupied classes among N deciders is $\sum_{i=1}^{N} \delta/(\delta + i - 1)$ (Escobar and West 1995), which gives about 3.9 classes for $N = 361$ at the prior mean $\delta = 0.5$ of the default hyperprior:

```
R> dynamic <- update(
+    electricity_classes, class_update = "dirichlet_process"
+  )
R> summary(dynamic, variables = c(
+    "weight[1]", "weight[2]", "weight[3]", "mu[loc,1]", "mu[loc,2]",
+    "mu[loc,3]", "class_concentration", "n_classes"
+  ))

Bayesian probit choice model
Formula: choice ~ pf + cl + loc + wk + tod + seas | 0 | 0
Samples: 750 retained per chain, 2 chains

            variable occupied   mean  mode     sd rhat ess_bulk
           weight[1]    1.000 0.8208 0.847 0.0622 1.00      365
           weight[2]    1.000 0.1615 0.144 0.0584 1.01      483
```

```
           weight[3]    0.531 0.0264 0.005 0.0421   NA       NA
           mu[loc,1]    1.000 1.5681 1.606 0.1372 1.00      726
           mu[loc,2]    1.000 4.9914 5.211 0.7529 1.00      567
           mu[loc,3]    0.531 3.6930 3.621 5.3445   NA       NA
 class_concentration    1.000 0.3906 0.266 0.2208 1.00     1038
           n_classes    1.000 2.8420 2.000 1.0226 1.00      397
```

Under the Dirichlet process, the number of occupied classes varies over the draws, and the column `occupied` reports the share of draws in which a class is occupied. The two classes of the finite mixture persist in every draw with similar weights and valuations, whereas a third class is occupied in about half of the draws. `latent_class_diagnostics()` summarizes this posterior through the distribution of the number of occupied classes and the membership probabilities of every household. The occupancy table places a posterior probability of 47 percent on two classes and 53 percent on three or more:

```
R> dynamic_diagnostics <- latent_class_diagnostics(dynamic)
R> subset(dynamic_diagnostics$occupancy, probability > 0)

  n_classes  probability
2         2 0.4686666667
3         3 0.3253333333
4         4 0.1340000000
5         5 0.0466666667
6         6 0.0186666667
7         7 0.0060000000
8         8 0.0006666667

R> head(dynamic_diagnostics$membership[, 1:3], 3)

    class_1    class_2     class_3
1 0.7973333 0.19000000 0.010666667
2 0.9613333 0.02866667 0.007333333
3 0.9740000 0.01800000 0.007333333
```

A sparse finite mixture addresses the same question within a finite model. It starts with more classes than needed, here `classes = 6`, and its default hyperprior with mean $\delta = 0.005$ favors small class weights, which empties the superfluous classes. The sparse mixture assigns a posterior probability of 91 percent to two occupied classes:

```
R> sparse <- update(
+    electricity_classes, class_update = "sparse", classes = 6
+  )
R> subset(latent_class_diagnostics(sparse)$occupancy, probability > 0)

  n_classes probability
2         2  0.90533333
3         3  0.09066667
4         4  0.00400000
```

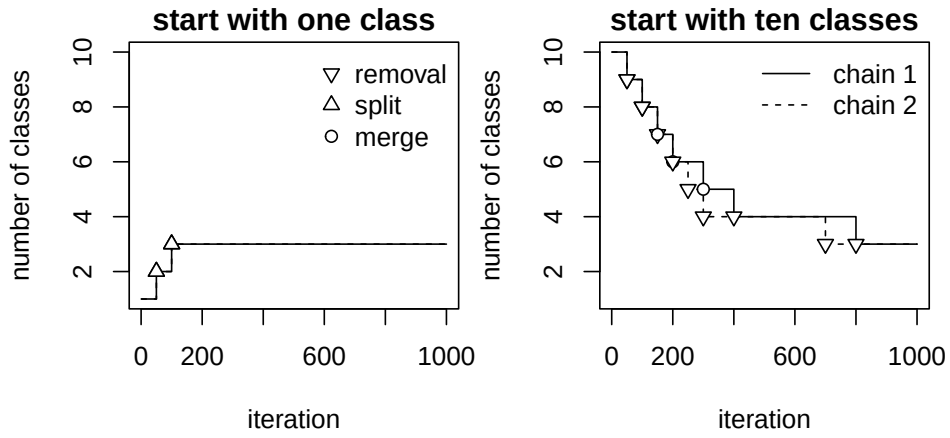


Figure 5: Number of classes during the first 1000 warmup iterations under the weight-based heuristic, started with one class (left) and with ten classes (right).

The fourth updating scheme is the heuristic of Oelschläger and Bauer (2021), requested by `class_update = "weight_based"`. During the warmup, it removes a class whose weight falls below `epsmin = 0.01`, splits a class whose weight exceeds `epsmax = 0.7`, and merges the two classes whose means are closer than `deltamin = 0.1`, at most one change per `buffer = 50` iterations. These values are the defaults of `weight_based_control`. After the warmup, the number of classes stays fixed. We demonstrate the heuristic on the simulation of Section 3.4, whose three classes are known, and start it once with a single class and once with ten classes to check whether the result depends on the initial number of classes:

```
R> heuristic_one <- update(
+    simulation, class_update = "weight_based", classes = 1
+  )
R> heuristic_ten <- update(
+    simulation, class_update = "weight_based", classes = 10
+  )
```

Figure 5 traces the number of classes during the warmup of both chains of the two runs. Started with one class, the heuristic splits it twice within the first 100 iterations. Started with ten classes, it mostly removes small classes and occasionally merges two. All four chains settle at three classes within the first 800 iterations. The weights and means of the classes found by the run from ten classes are close to those of the DGP, and the run from a single class yields a similar result:

```
R> summary(heuristic_ten, variables = c(
+    "weight[1]", "weight[2]", "weight[3]", "mu[price,1]", "mu[price,2]",
+    "mu[price,3]", "n_classes"
+  ))

Bayesian probit choice model
Formula: choice ~ price | income | time
Samples: 1000 retained per chain, 2 chains
```

```
    variable     mean    mode     sd rhat ess_bulk
   weight[1]  0.35981  0.3608 0.0245 1.00     1570
   weight[2]  0.32636  0.3324 0.0241 1.00     1458
   weight[3]  0.31383  0.3066 0.0241 1.00     1358
 mu[price,1] -1.83440 -1.8348 0.0649 1.01      313
 mu[price,2]  1.87510  1.8541 0.0659 1.00      391
 mu[price,3]  0.00991  0.0112 0.0393 1.00     1034
   n_classes  3.00000  3.0000 0.0000   NA       NA
```

6. Ordered and ranked responses

`fit()` estimates the models of Section 2.6 with `choice_type = "ordered"` or `"ranked"`.

6.1. Ordered responses

We model the self-reported health satisfaction of adults as a function of age, income, the degree of handicap in percent (`handper`), and marital status, and quantify its persistent person-specific component. The `Health` data of the **Rchoice** package, an extract of the German Socio-Economic Panel compiled by Riphahn *et al.* (2003), comprise 27326 observations of 7293 persons from seven survey years between 1984 and 1994. The response `newhsat` records the satisfaction with health on five ordered levels, from 0 for the lowest to 4 for the highest satisfaction, which correspond to $j = 1, \dots, 5$ in (10). A normal random effect on the intercept θ_n^0 captures the between-person variation of the baseline satisfaction. Age and monthly household net income in thousand marks are centered at their sample means.

```
R> data("Health", package = "Rchoice")
R> Health$age <- Health$age - mean(Health$age)
R> Health$income <- Health$hhinc / 1000 - mean(Health$hhinc / 1000)
R> Health$occasion <- with(Health, ave(id, id, FUN = seq_along))
R> satisfaction <- fit(
+    newhsat ~ age + income + handper + married, data = Health,
+    alternatives = as.character(0:4), choice_type = "ordered",
+    random_effects = "ASC", column_decider = "id",
+    column_occasion = "occasion",
+    iterations = 20000, warmup = 10000, thin = 10, chains = 2
+  )
R> summary(satisfaction)

Bayesian probit choice model
Formula: newhsat ~ age + income + handper + married | 1 | 0
Samples: 1000 retained per chain, 2 chains

       variable    mean    mode       sd  rhat ess_bulk
      beta[age] -0.0283 -0.0281 0.001141 1.001     1552
   beta[income]  0.0160  0.0160 0.005486 0.999     2022
```

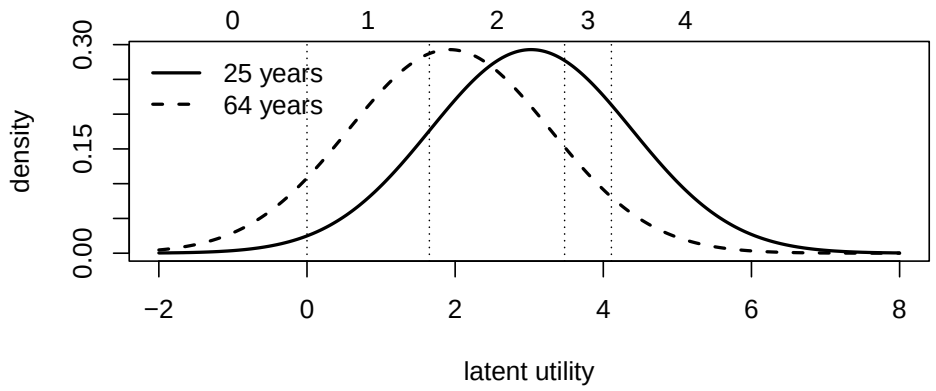


Figure 6: Density of the latent utility of a married person with mean income and no handicap at 25 years (solid) and 64 years (dashed) of age. The dotted lines mark the thresholds $\gamma_1 = 0$ and γ_2 to γ_4, and the areas under a density between the thresholds are the probabilities of the five levels.

```
beta[handper] -0.0131 -0.0130 0.000581 1.000     1855
beta[married]  0.0522  0.0516 0.026358 1.000      703
      mu[ASC]  2.4491  2.4495 0.029967 1.000      393
Omega[ASC,ASC] 0.8558  0.8486 0.025700 1.001     1139
      gamma[2] 1.6520  1.6531 0.019303 1.000      428
      gamma[3] 3.4776  3.4731 0.024177 1.001      458
      gamma[4] 4.1093  4.1061 0.026485 1.001      500
```

The summary lists the coefficients, the free thresholds `gamma[2]` to `gamma[4]`, and the mean and the variance of the random intercept, `mu[ASC]` and `Omega[ASC,ASC]`. The first threshold is fixed at zero and the error variance at one. According to the model, the probability of a higher satisfaction level decreases with age and with the degree of handicap and increases with income and for married persons. With a random intercept variance of 0.86 and an error variance of one, persistent differences between persons account for about 46 percent of the unexplained variation in the latent utility. The repeated responses of the same persons identify these differences, which capture determinants that the covariates omit, such as chronic conditions or individual standards of judging one's health. Figure 6 shows how the thresholds translate the latent utility of a married person with mean income and no handicap into the five levels: an age of 64 instead of 25 years shifts the density to the left and moves probability from the two highest to the two lowest levels. The thresholds are unequally spaced: level 2 covers the widest bounded interval of the latent utility and is the most probable level at both ages, whereas level 3 covers a narrow interval and is less probable than the highest level at 25 years. Section 7.2 predicts the probability of the highest level over the age range of the sample.

6.2. Ranked responses

The `Game` data of the **mlogit** package (Fok *et al.* 2012) record the preferences of 91 Dutch respondents for six gaming platforms: Xbox, PlayStation, PSPortable, GameCube, GameBoy, and PC. Every respondent ranks all six platforms from the most to the least preferred.

Platform ownership (`own`) enters the formula as a covariate of type `A`, and the weekly hours of gaming (`hours`) and the age of the respondent (`age`) as covariates of type `B`, whose coefficients are interpreted relative to the GameBoy as the base alternative. The respondents are between 17 and 33 years old, with a median of 20 years, and play a median of 2 hours per week. Almost all of them own a PC (80 of 91), whereas between 8 and 29 respondents own each of the other platforms, and half of the respondents own a single platform. The PC is ranked first by 39 respondents, the PlayStation and the Xbox by 18 each, the GameCube and the PSPortable by 7 each, and the GameBoy by only 2. For the comparison at the end of this section, we store the first-ranked platform of every respondent in the column `first`:

```
R> data("Game", package = "mlogit")
R> rank_columns <- grep("^ch\\.", names(Game), value = TRUE)
R> Game$first <- sub("ch.", "", rank_columns, fixed = TRUE)[
+    apply(Game[rank_columns], 1, which.min)
+  ]
R> gaming_rank <- fit(
+    ch ~ own | hours + age, data = Game, choice_type = "ranked",
+    delimiter = ".", column_decider = NULL,
+    iterations = 40000, warmup = 20000, thin = 20, chains = 2
+  )
```

Figure 7 shows the marginal posteriors of the ownership, hours, and age coefficients, drawn by `plot()` with `type = "interval"`:

```
R> plot(
+    gaming_rank, type = "interval", prob_outer = 0.95,
+    variables = grep(
+      "own|hours|age", names(coef(gaming_rank)), value = TRUE
+    )
+  )
```

According to the model, respondents rank the platforms they own higher than those they do not own. The ownership coefficient has a posterior mean of 0.85 and a 95% credible interval from 0.53 to 1.20. Respondents who play more hours rank the PC and, to a lesser extent, the PlayStation higher relative to the GameBoy: the posterior mean of the hours coefficient is 0.20 for the PC and 0.07 for the PlayStation, that is, one more hour of gaming per week raises their utility relative to the GameBoy by these amounts, whereas it lies between -0.01 and 0.04 for the other three platforms. Older respondents rank the PC slightly higher, with a posterior mean of 0.06 per year for its age coefficient and between 0.00 and 0.03 for the other platforms.

To quantify the information that the complete rankings add to the first choices (Section 2.6), we fit the same specification to the first-ranked platforms as an unordered choice:

```
R> gaming_first <- update(
+    gaming_rank, first ~ own | hours + age, choice_type = "unordered"
+  )
```

The posterior means and standard deviations of the coefficients under both models are:

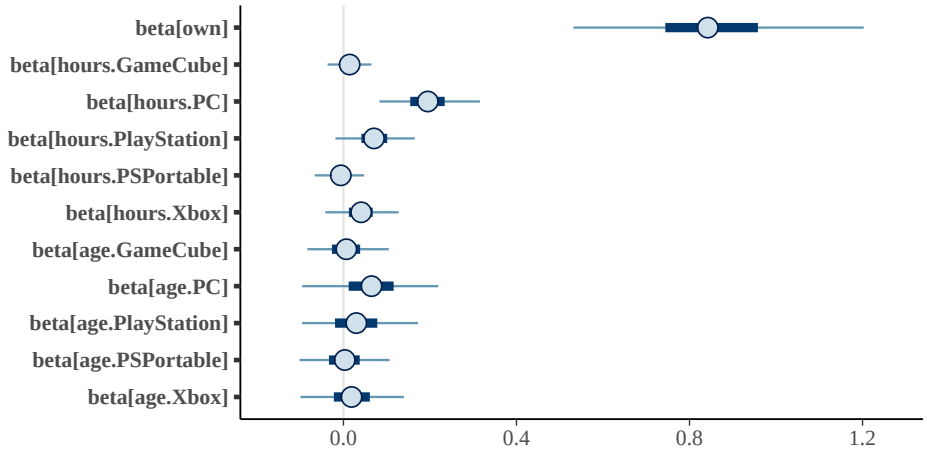


Figure 7: Marginal posterior distributions of the ownership coefficient and of the hours and age coefficients relative to the GameBoy in the ranked model `gaming_rank`: posterior medians (points), 50% credible intervals (thick lines), and 95% credible intervals (thin lines).

```
R> gaming_comparison <- merge(
+    summary(gaming_rank)$posterior[, c("variable", "mean", "sd")],
+    summary(gaming_first)$posterior[, c("variable", "mean", "sd")],
+    by = "variable", suffixes = c("_ranked", "_first")
+  )
R> print(
+    subset(gaming_comparison, grepl("^beta", variable)),
+    digits = 2, row.names = FALSE
+  )
```

variable	mean_ranked	sd_ranked	mean_first	sd_first
beta[age.GameCube]	0.0070	0.048	-0.220	0.112
beta[age.PC]	0.0638	0.079	0.035	0.091
beta[age.PlayStation]	0.0303	0.071	0.088	0.089
beta[age.PSPortable]	0.0027	0.054	-0.040	0.101
beta[age.Xbox]	0.0197	0.062	-0.064	0.097
beta[ASC.GameCube]	0.0021	0.966	4.342	2.204
beta[ASC.PC]	-0.9230	1.597	-1.291	1.826
beta[ASC.PlayStation]	0.3967	1.416	-1.854	1.821
beta[ASC.PSPortable]	0.6196	1.083	0.226	1.947
beta[ASC.Xbox]	0.9236	1.246	1.402	1.928
beta[hours.GameCube]	0.0141	0.026	-0.147	0.133
beta[hours.PC]	0.1951	0.060	0.146	0.093
beta[hours.PlayStation]	0.0712	0.047	0.030	0.084
beta[hours.PSPortable]	-0.0069	0.029	0.039	0.116
beta[hours.Xbox]	0.0410	0.042	0.034	0.086
beta[own]	0.8498	0.170	1.272	0.383

Although both models use the responses of the same 91 respondents, the posterior standard

deviations are up to 5 times larger with the first choices than with the rankings. The hours effect of the PC, whose 95% credible interval excludes zero under the rankings, is no longer distinguishable from zero under the first choices. In this application, the complete rankings thus yield considerably more precise estimates than the first choices alone.

7. Prediction and residuals

The `predict()` method computes posterior predictive choice probabilities, whose deviations from the observed choices define the residuals of Section 7.3. The population choice probability integrates the choice probability of Section 2.7 over the population distribution of θ_n,

$$\mathsf{P}(y_{nt} = j \mid X_{nt}, \vartheta) = \int \mathsf{P}(y_{nt} = j \mid X_{nt}, \theta_n, \tilde{\Sigma})\, p(\theta_n \mid \vartheta)\, \mathrm{d}\theta_n, \tag{21}$$

where $p(\theta_n \mid \vartheta)$ is the distribution of θ_n implied by (5), (7), and (8). The population posterior predictive probability averages (21) over the posterior,

$$\mathsf{P}^{\text{pop}}(y_{nt} = j \mid X_{nt}, y) = \int \mathsf{P}(y_{nt} = j \mid X_{nt}, \vartheta)\, p(\vartheta \mid y)\, \mathrm{d}\vartheta \approx \frac{1}{D} \sum_{d=1}^{D} \mathsf{P}\big(y_{nt} = j \mid X_{nt}, \vartheta^{(d)}\big). \tag{22}$$

7.1. Population and conditional predictions

For a decider n in the sample, `type = "conditional"` replaces the population distribution by the posterior draws of the decider's own coefficients and class allocation (Section 5.2),

$$\mathsf{P}^{\text{cond}}(y_{nt} = j \mid X_{nt}, y) \approx \frac{1}{D} \sum_{d=1}^{D} \mathsf{P}\big(y_{nt} = j \mid X_{nt}, \theta_n^{(d)}, \tilde{\Sigma}^{(d)}\big), \tag{23}$$

and thus exploits the choice history of the decider.

We compare both types of prediction for a repeated binary decision in an online chess tournament. In arena tournaments on the chess server Lichess, a player may go "Berserk" at the start of every game: the player's clock is halved, and a win earns one extra tournament point. Going Berserk is thus a gamble that trades a higher risk of losing for a larger reward in case of a win. The `lichess_berserk_choice` data of **choicedata** record this decision for the players of the Lichess Yearly Rapid Arena of April 2026, together with the playing color, the player's rating, the rating difference to the opponent, the remaining tournament time, and whether the player was on a winning streak, which doubles the points of a win. We standardize the three numeric covariates, and `train_test()` holds out one randomly chosen game per player:

```
R> data("lichess_berserk_choice", package = "choicedata")
R> berserk_data <- as.data.frame(lichess_berserk_choice)
R> berserk_data <- transform(
+    berserk_data, rating = c(scale(rating)),
+    ratingDifference = c(scale(ratingDifference)),
+    minutesRemaining = c(scale(minutesRemaining))
+  )
```

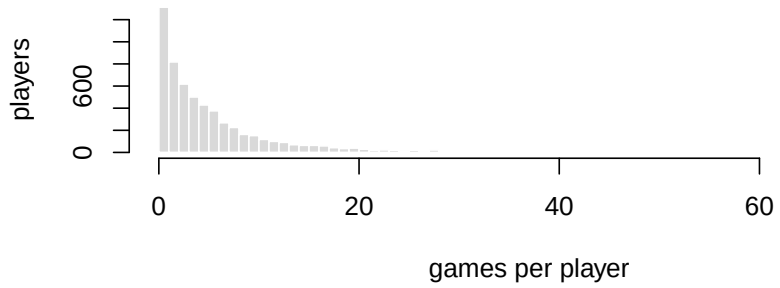


Figure 8: Number of games per player in the `lichess_berserk_choice` data.

```
R> berserk_split <- train_test(
+    berserk_data, test_number = 1, by = "occasion", random = TRUE,
+    column_occasion = "occasionID"
+  )
R> sapply(berserk_split, nrow)

train  test
31564  5852
```

The data contain 37416 games of 5852 players, between 1 and 74 per player. Figure 8 shows that most players play only a few games, with a median of 4 games per player. Players may differ in their inclination to go Berserk beyond what the covariates explain. We therefore fit the model without and with a random effect on the ASC, which represents this inclination as a normally distributed propensity of every player, and save the individual draws for the conditional predictions:

```
R> berserk_fixed <- fit(
+    berserk ~ 0 | white + rating + ratingDifference + minutesRemaining +
+      streak,
+    data = berserk_split$train, column_occasion = "occasionID",
+    iterations = 1000, warmup = 500, chains = 2
+  )
R> berserk <- update(
+    berserk_fixed, random_effects = "ASC", iterations = 10000,
+    warmup = 5000, thin = 50, save_individual_draws = TRUE
+  )
R> round(cbind(mean = coef(berserk), confint(berserk, level = 0.9)), 3)

                              mean     5%    95%
beta[whiteTRUE_TRUE]         0.054  0.012  0.097
beta[rating_TRUE]            0.244  0.178  0.315
beta[ratingDifference_TRUE]  0.679  0.653  0.705
beta[minutesRemaining_TRUE] -0.179 -0.211 -0.152
beta[streakTRUE_TRUE]       -0.007 -0.061  0.044
mu[ASC_TRUE]                -1.917 -2.002 -1.834
Omega[ASC_TRUE,ASC_TRUE]     4.737  4.379  5.129
```

According to the model, players go Berserk more often with the white pieces, which have the advantage of the first move, when their rating is high, when it exceeds that of the opponent, and late in the tournament, while a winning streak has no clear effect. Among the standardized covariates, the rating difference clearly has the strongest effect. The negative mean `mu[ASC_TRUE]` of the propensity reflects that 74 percent of the games are played without Berserk, and the propensity varies considerably between players, as indicated by the variance `Omega[ASC_TRUE,ASC_TRUE]` of about 4.7. The model without a random effect predicts each hold-out game by its population probability, whereas the conditional prediction of `berserk` uses the player's other games:

```
R> known <- berserk_split$test$deciderID %in%
+    berserk_split$train$deciderID
R> holdout <- berserk_split$test[known, ]
R> holdout_fixed <- predict(berserk_fixed, newdata = holdout)
R> holdout_conditional <- predict(
+    berserk, newdata = holdout, type = "conditional"
+  )
R> head(holdout_conditional, 3)
```

```
  deciderID occasionID .prediction probability_FALSE probability_TRUE
1 A_A_Y_N_A          3       FALSE         0.9439363       0.05606375
2 A_Erikson          5       FALSE         0.9319100       0.06808996
3    A_Tako          8       FALSE         0.9838692       0.01613080
```

`predict()` returns a data frame with one row per choice occasion, with the most probable alternative in `.prediction` and the posterior predictive probabilities in `probability_*`. Excluding the 1313 players with a single game leaves 4539 hold-out games. For these games, the share of correct predictions is 78.3 percent for the model without a random effect, 78.1 percent for the naive rule that never predicts Berserk, and 88.8 percent for the conditional predictions. Figure 9 shows the receiver operating characteristic (ROC) curves of both predictions, drawn with **plotROC** (Sachs 2017) and **ggplot2** (Wickham 2016). The area under the curve is 0.92 for the conditional predictions and 0.68 for the model without a random effect. The conditional predictions are thus considerably better.

7.2. Predictions for new covariate values

With `newdata`, `predict()` computes the probabilities at arbitrary covariate values, and `uncertainty = TRUE` adds the posterior standard deviation and an equal-tailed credible interval of every probability, at the default level of 95%, which the argument `level` changes. For the ordered model `satisfaction` of Section 6.1, we predict the satisfaction of a married person with mean income over the age range of the sample, without a handicap and with a handicap of 50 percent. These are population predictions, which integrate over the random intercept. Since age is centered, the grid value zero is the mean age of 43.5 years, and the sequence covers the ages from 25 to 64 years in steps of one year. The suffix of the columns `probability_4`, `lower_4`, and `upper_4` refers to the highest level. The last command prints its probability and credible interval at the mean age, without and with the handicap:

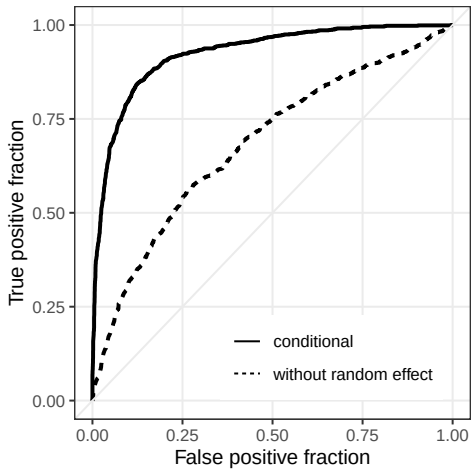


Figure 9: Receiver operating characteristic curves for the hold-out games of the players with at least two games: conditional predictions of the model with a random effect on the ASC (solid) and population predictions of the model without a random effect (dashed).

```
R> health_grid <- expand.grid(
+    age = c(0, seq(min(Health$age), max(Health$age))), income = 0,
+    handper = c(0, 50), married = 1
+  )
R> grid_prediction <- predict(
+    satisfaction, newdata = health_grid, uncertainty = TRUE
+  )
R> grid_prediction <- cbind(health_grid, grid_prediction)
R> subset(
+    grid_prediction, age == 0,
+    c(handper, probability_4, lower_4, upper_4)
+  )

   handper probability_4    lower_4    upper_4
1        0    0.11894194 0.11341335 0.12474313
42      50    0.04846994 0.04366832 0.05336274
```

At the mean age and income, the posterior predictive probability of the highest level is 0.12 without and 0.05 with a handicap of 50 percent. Figure 10 shows the probabilities and their credible intervals over the age range. Without a handicap, the probability falls from 0.21 at 25 years to 0.05 at 64 years, and the handicap reduces it by more than half at every age.

7.3. Residuals

The `residuals()` method returns the observed choice indicators minus the posterior predictive probabilities of (22),

$$e_{ntj} = 1(y_{nt} = j) - \mathsf{P}^{\text{pop}}(y_{nt} = j \mid X_{nt}, y), \qquad j = 1, \ldots, J, \tag{24}$$

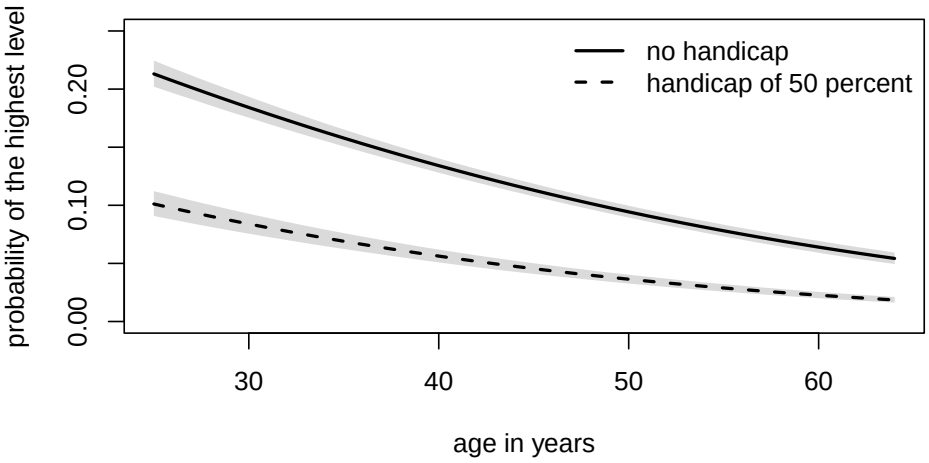


Figure 10: Posterior predictive probability of the highest satisfaction level by age for a married person with mean income, without a handicap (solid) and with a handicap of 50 percent (dashed), with 95% credible intervals (shaded).

as a matrix with one row per choice occasion and one column per alternative. The residuals of an occasion sum to zero, and with two alternatives, they differ only in sign. A small residual of the chosen alternative indicates an accurately predicted choice, whereas an absolute value close to one indicates a choice that the model deems unlikely. The row names of the residual matrix combine the decider and the choice occasion. For the first three choices of the first traveler in `model_train`, the residuals are:

```
R> train_residuals <- residuals(model_train)
R> head(train_residuals, 3)
```

```
              A           B
1:1 0.07770688 -0.07770688
1:2 0.35932006 -0.35932006
1:3 0.15323619 -0.15323619
```

8. Model evaluation

For the comparison of competing specifications, **RprobitB** provides three criteria: the widely applicable information criterion (WAIC) of Watanabe (2010), leave-one-out cross-validation approximated by Pareto-smoothed importance sampling (PSIS-LOO) of Vehtari *et al.* (2017, 2024), and Bayes factors (Kass and Raftery 1995). The first two estimate the expected predictive accuracy of a model for new deciders, whereas the third compares the marginal likelihoods of two models. We write $p_n(\vartheta) = p(y_n \mid X_n, \vartheta)$ for the likelihood contribution of decider n. Since WAIC and PSIS-LOO evaluate p_n under every retained draw, their cost grows with the number of draws and deciders. Thinning or a parallel **future** plan can be used to reduce the computational load.

We compare the three train models in `models` (Section 5.3), whose price coefficient is fixed in `model_train`, normal in `price_normal`, and negative log-normal in `price_lognormal`.

8.1. Log-likelihood

`logLik()` evaluates the log-likelihood $\sum_n \log p_n(\bar{\vartheta})$ at the posterior means $\bar{\vartheta}$ of the population parameters and returns a '`logLik`' object, which `AIC()` and `BIC()` accept:

```
R> sapply(models, logLik)
```

```
    model_train    price_normal price_lognormal
      -1698.796       -1539.986       -1545.775
```

The random price coefficient, which adds one free parameter to the five of `model_train`, raises the log-likelihood by 159 under the normal and by 153 under the negative log-normal mixing distribution. Since `logLik()`, `AIC()`, and `BIC()` rely on a single point estimate, they ignore the posterior uncertainty about ϑ, unlike WAIC and PSIS-LOO.

8.2. WAIC and PSIS-LOO

Both criteria estimate the expected log predictive density (elpd) of a model for the responses $\tilde{y}_n$ of new deciders,

$$\mathrm{elpd} = \sum_{n=1}^{N} \mathbb{E}\big[\log p(\tilde{y}_n \mid X_n, y)\big], \qquad p(\tilde{y}_n \mid X_n, y) = \int p(\tilde{y}_n \mid X_n, \vartheta)\, p(\vartheta \mid y)\, \mathrm{d}\vartheta, \tag{25}$$

which increases with the predictive accuracy. WAIC estimates the elpd as

$$\widehat{\mathrm{elpd}}_{\mathrm{waic}} = \sum_{n=1}^{N} \log\left(\frac{1}{D}\sum_{d=1}^{D} p_n(\vartheta^{(d)})\right) - \sum_{n=1}^{N} \mathrm{Var}_d\big[\log p_n(\vartheta^{(d)})\big], \qquad \mathrm{WAIC} = -2\,\widehat{\mathrm{elpd}}_{\mathrm{waic}}, \tag{26}$$

where Var_d is the sample variance over the D draws. PSIS-LOO instead estimates the leave-one-out predictive probability $p(y_n \mid y_{-n})$ of the responses of decider n given the responses y_{-n} of all other deciders by importance sampling over the retained draws,

$$\widehat{\mathrm{elpd}}_{\mathrm{loo}} = \sum_{n=1}^{N} \log \hat{p}(y_n \mid y_{-n}), \qquad \hat{p}(y_n \mid y_{-n}) = \frac{\sum_{d=1}^{D} w_n^{(d)}\, p_n(\vartheta^{(d)})}{\sum_{d=1}^{D} w_n^{(d)}}, \tag{27}$$

where the weights $w_n^{(d)}$ are Pareto-smoothed importance ratios. Both estimates are sums of pointwise contributions $\widehat{\mathrm{elpd}}_n$. The standard error of the difference between two models $\mathcal{M}_1$ and $\mathcal{M}_2$ follows from the variance of the pointwise differences,

$$\Delta_n = \widehat{\mathrm{elpd}}_n(\mathcal{M}_1) - \widehat{\mathrm{elpd}}_n(\mathcal{M}_2), \qquad \mathrm{se}\Big(\sum_{n=1}^{N} \Delta_n\Big) = \sqrt{N\,\mathrm{Var}_n(\Delta_n)}, \tag{28}$$

with Var_n the sample variance over the N deciders (Vehtari *et al.* 2017). `WAIC()` and `loo()` pass the $D \times N$ matrix of the decider log-likelihoods to the **loo** package and return the objects of that package. For `model_train`, `WAIC()` gives $\widehat{\mathrm{elpd}}_{\mathrm{waic}} = -1707.0$ with a standard error of 40.2. For four of the travelers, the variance term exceeds 0.4, the level above which Vehtari *et al.* (2017) consider the WAIC approximation unreliable. PSIS-LOO is then preferable, because it is more robust in such cases. `loo_compare()` ranks the three models by $\widehat{\mathrm{elpd}}_{\mathrm{loo}}$:

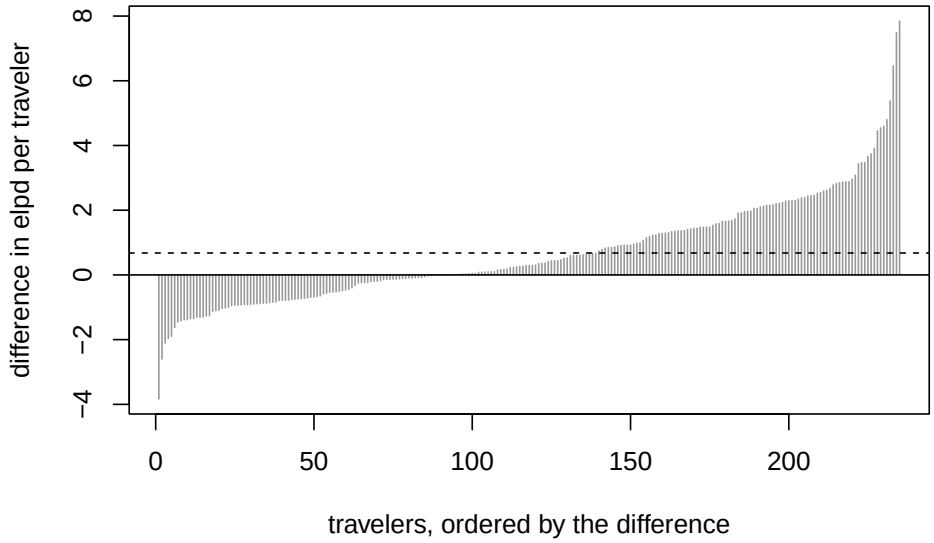


Figure 11: Pointwise differences Δ_n of (28) in the leave-one-out expected log predictive density between `price_normal` and `model_train`, one vertical line per traveler, ordered by size. Positive values favor `price_normal`. The dashed line marks the mean difference.

```
R> train_loo <- lapply(models, loo)
R> loo_comparison <- loo::loo_compare(train_loo)
R> loo_comparison
```

```
           model elpd_diff se_diff p_worse diag_diff diag_elpd
    price_normal       0.0     0.0      NA
 price_lognormal      -4.3    10.0    0.67
     model_train    -159.2    25.1    1.00
```

The column `p_worse` gives the probability that a model has a lower elpd than the first-ranked one. The normal mixing distribution ranks first, but the elpd under the negative log-normal mixing distribution is lower by only 4.3 with a standard error of 10.0. The data therefore do not clearly favor either mixing distribution. Under a fixed price coefficient, in contrast, the elpd is lower by 159 with a standard error of 25. A random price coefficient thus clearly improves the prediction of a traveler's choices. According to Figure 11, the random price coefficient raises the elpd contribution of 60 percent of the travelers. The same comparison applies to the number of latent classes. For the `Electricity` data, we compare the model `electricity` of Section 5.1, in which the valuation of a local supplier follows a single normal distribution, with the two-class model `electricity_classes` of Section 5.4:

```
R> classes_loo <- lapply(mget(c("electricity", "electricity_classes")), loo)
R> classes_comparison <- loo::loo_compare(classes_loo)
R> classes_comparison
```

Running this code takes more than ten minutes. The elpd of the one-class model is lower by 16.8 with a standard error of 5.2, which favors the two classes.

8.3. Bayes factors

A Bayes factor compares two models $\mathcal{M}_1$ and $\mathcal{M}_2$ by the ratio of their marginal likelihoods,

$$\mathrm{BF}_{12} = \frac{p(y \mid \mathcal{M}_1)}{p(y \mid \mathcal{M}_2)}, \qquad p(y \mid \mathcal{M}) = \int \prod_{n=1}^{N} p(y_n \mid X_n, \vartheta, \mathcal{M})\, p(\vartheta \mid \mathcal{M})\, \mathrm{d}\vartheta, \tag{29}$$

where $p(\vartheta \mid \mathcal{M})$ is the prior of Section 3.2. Kass and Raftery (1995) interpret values of $2 \log \mathrm{BF}_{12}$ above 10 as very strong evidence for $\mathcal{M}_1$. `bayes_factor()` estimates the marginal likelihoods by bridge sampling (Meng and Wong 1996; Frühwirth-Schnatter 2004; Gronau *et al.* 2017).

```
R> log_bf <- bayes_factor(price_normal, model_train, log = TRUE)
R> log_bf

Estimated log Bayes factor in favor of model1 over model2: 156.75518
```

The log Bayes factor of 157, that is, $2 \log \mathrm{BF}_{12} \approx 314$, is very strong evidence for the normal mixing distribution over the fixed price coefficient, in line with the comparison by PSIS-LOO.

9. Summary

The R package **RprobitB** implements the Bayesian estimation of probit models for binary, multinomial, ordered, and ranked responses. All models are specified through the single function `fit()`, whose formula interface distinguishes the three covariate types of discrete choice data. The error covariance is estimated, which permits flexible substitution patterns, and preference heterogeneity is modeled through random coefficients with normal or log-normal mixing distributions, through latent classes, or through both. The number of latent classes can be fixed in advance or inferred from the data. For efficient computation, the Gibbs sampler is implemented in C++ and can run several chains in parallel.

Since the derived quantities are evaluated under all retained draws, the posterior uncertainty propagates to willingness-to-pay measures, marginal effects, predicted choice probabilities, and the coefficients of individual deciders. Conditional predictions incorporate the choice history of a decider, and the posterior distribution of the number of latent classes quantifies the uncertainty about the number of segments. Together with the standard methods of R and the model comparison, the package covers a full analysis workflow.

The package is intended for applied researchers and practitioners in transportation, marketing, health, and related fields who analyze discrete choice data and who report willingness-to-pay measures, marginal effects, or predictions together with their uncertainty. Suggestions are welcome as issues at https://github.com/loelschlaeger/RprobitB/issues.

Computational details

The results in this paper were obtained using R 4.6.1 with the packages **RprobitB** 2.0.1 and **choicedata** 0.2.1 under macOS Tahoe 26.6.2. The data sets of the examples come from **mlogit** 2.0.0, **Rchoice** 0.3.6, and **choicedata**. **RprobitB** builds on **oeli** 0.7.9 (Oelschläger

2026c), **Rcpp** 1.1.2, and **RcppArmadillo** 15.6.0.1 for the sampler and on **Formula** 1.2.6 for the formula interface. The retained draws are stored with **posterior** 1.7.0 and displayed with **bayesplot** 1.16.0, and model evaluation uses **loo** 2.10.1 and **bridgesampling** 1.2.1. The ROC curves are drawn with **plotROC** 2.3.3 and **ggplot2** 4.0.3. Parallel chains and progress reporting use **future** 1.75.0, **future.apply** 1.20.2 (Bengtsson 2026a), and **progressr** 1.0.0 (Bengtsson 2026b). The chains and the likelihood evaluations of `loo()` ran in parallel on two workers under a multisession plan of **future**. R itself and all packages used are available from the Comprehensive R Archive Network (CRAN) at https://CRAN.R-project.org/.

References

Albert JH, Chib S (1993). "Bayesian Analysis of Binary and Polychotomous Response Data." *Journal of the American Statistical Association*, **88**(422), 669–679. doi:10.1080/01621459.1993.10476321.

Allenby GM, Rossi PE (1998). "Marketing Models of Consumer Heterogeneity." *Journal of Econometrics*, **89**(1-2), 57–78. doi:10.1016/S0304-4076(98)00055-4.

Bauer D, Batram M, Büscher S, Oelschläger L (2024). ***Rprobit**: Estimation of Multinomial Probit Models*. R package version 0.3.4, URL https://github.com/dbauer72/Rprobit.

Ben-Akiva M, Bolduc D, Bradley M (1993). "Estimation of Travel Choice Models with Randomly Distributed Values of Time." *Transportation Research Record*, **1413**, 88–97. URL https://trid.trb.org/View/385096.

Bengtsson H (2021). "A Unifying Framework for Parallel and Distributed Processing in R Using Futures." *The R Journal*, **13**(2), 208–227. doi:10.32614/RJ-2021-048.

Bengtsson H (2026a). ***future.apply**: Apply Function to Elements in Parallel Using Futures*. doi:10.32614/CRAN.package.future.apply. R package version 1.20.2.

Bengtsson H (2026b). ***progressr**: An Inclusive, Unifying API for Progress Updates*. doi:10.32614/CRAN.package.progressr. R package version 1.0.0.

Bhat CR (2011). "The Maximum Approximate Composite Marginal Likelihood (MACML) Estimation of Multinomial Probit-Based Unordered Response Choice Models." *Transportation Research Part B: Methodological*, **45**(7), 923–939. doi:10.1016/j.trb.2011.04.005.

Bierlaire M (2020). "A Short Introduction to **PandasBiogeme**." *Technical Report TRANSP-OR 200605*, Transport and Mobility Laboratory, ENAC, École Polytechnique Fédérale de Lausanne, Lausanne, Switzerland. URL https://transp-or.epfl.ch/documents/technicalReports/Bier20.pdf.

Bierlaire M (2025). "Bayesian Estimation with **Biogeme**: User Guide." *Technical Report TRANSP-OR 251225*, Transport and Mobility Laboratory, ENAC, École Polytechnique Fédérale de Lausanne, Lausanne, Switzerland. URL https://transp-or.epfl.ch/documents/technicalReports/Bier25_bayes_b.pdf.

Brand BM, Baier D (2022). "Measuring Country of Origin Effects in Online Shopping Implicitly: A Discrete Choice Analysis Approach." *International Marketing Review*, **39**(4), 955–983. doi:10.1108/IMR-03-2021-0139.

Bürkner PC, Gabry J, Kay M, Vehtari A (2026). ***posterior**: Tools for Working with Posterior Distributions*. doi:10.32614/CRAN.package.posterior. R package version 1.7.0.

Caputo V, Scarpa R, Nayga RM, Ortega DL (2018). "Are Preferences for Food Quality Attributes Really Normally Distributed? An Analysis Using Flexible Mixing Distributions." *Journal of Choice Modelling*, **28**, 10–27. doi:10.1016/j.jocm.2018.04.003.

Cirillo C, Axhausen KW (2006). "Evidence on the Distribution of Values of Travel Time Savings from a Six-Week Diary." *Transportation Research Part A: Policy and Practice*, **40**(5), 444–457. doi:10.1016/j.tra.2005.06.007.

Croissant Y (2020). "Estimation of Random Utility Models in R: The **mlogit** Package." *Journal of Statistical Software*, **95**(11), 1–41. doi:10.18637/jss.v095.i11.

Dahl DB (2006). "Model-Based Clustering for Expression Data via a Dirichlet Process Mixture Model." In KA Do, P Müller, M Vannucci (eds.), *Bayesian Inference for Gene Expression and Proteomics*, pp. 201–218. Cambridge University Press. doi:10.1017/CBO9780511584589.011.

Eddelbuettel D, François R (2011). "**Rcpp**: Seamless R and C++ Integration." *Journal of Statistical Software*, **40**(8), 1–18. doi:10.18637/jss.v040.i08.

Eddelbuettel D, Sanderson C (2014). "**RcppArmadillo**: Accelerating R with High-Performance C++ Linear Algebra." *Computational Statistics & Data Analysis*, **71**, 1054–1063. doi:10.1016/j.csda.2013.02.005.

Escobar MD, West M (1995). "Bayesian Density Estimation and Inference Using Mixtures." *Journal of the American Statistical Association*, **90**(430), 577–588. doi:10.1080/01621459.1995.10476550.

Fok D, Paap R, van Dijk B (2012). "A Rank-Ordered Logit Model with Unobserved Heterogeneity in Ranking Capabilities." *Journal of Applied Econometrics*, **27**(5), 831–846. doi:10.1002/jae.1223.

Fosgerau M (2006). "Investigating the Distribution of the Value of Travel Time Savings." *Transportation Research Part B: Methodological*, **40**(8), 688–707. doi:10.1016/j.trb.2005.09.007.

Frühwirth-Schnatter S (2004). "Estimating Marginal Likelihoods for Mixture and Markov Switching Models Using Bridge Sampling Techniques." *The Econometrics Journal*, **7**(1), 143–167. doi:10.1111/j.1368-423X.2004.00125.x.

Frühwirth-Schnatter S, Malsiner-Walli G (2019). "From Here to Infinity: Sparse Finite versus Dirichlet Process Mixtures in Model-Based Clustering." *Advances in Data Analysis and Classification*, **13**(1), 33–64. doi:10.1007/s11634-018-0329-y.

Gabry J, Mahr T (2026). ***bayesplot**: Plotting for Bayesian Models*. doi:10.32614/CRAN.package.bayesplot. R package version 1.16.0.

Greene WH, Hensher DA (2003). "A Latent Class Model for Discrete Choice Analysis: Contrasts with Mixed Logit." *Transportation Research Part B: Methodological*, **37**(8), 681–698. doi:10.1016/S0191-2615(02)00046-2.

Gronau QF, Sarafoglou A, Matzke D, Ly A, Boehm U, Marsman M, Leslie DS, Forster JJ, Wagenmakers EJ, Steingroever H (2017). "A Tutorial on Bridge Sampling." *Journal of Mathematical Psychology*, **81**, 80–97. doi:10.1016/j.jmp.2017.09.005.

Gronau QF, Singmann H, Wagenmakers EJ (2020). "**bridgesampling**: An R Package for Estimating Normalizing Constants." *Journal of Statistical Software*, **92**(10), 1–29. doi:10.18637/jss.v092.i10.

Hajivassiliou VA, McFadden DL, Ruud PA (1996). "Simulation of Multivariate Normal Rectangle Probabilities and Their Derivatives: Theoretical and Computational Results." *Journal of Econometrics*, **72**(1-2), 85–134. doi:10.1016/0304-4076(94)01716-6.

Hausman JA, Wise DA (1978). "A Conditional Probit Model for Qualitative Choice: Discrete Decisions Recognizing Interdependence and Heterogeneous Preferences." *Econometrica*, **46**(2), 403–426. doi:10.2307/1913909.

Helveston JP (2023). "**logitr**: Fast Estimation of Multinomial and Mixed Logit Models with Preference Space and Willingness-to-Pay Space Utility Parameterizations." *Journal of Statistical Software*, **105**(10), 1–37. doi:10.18637/jss.v105.i10.

Helveston JP, Liu Y, Feit EM, Fuchs E, Klampfl E, Michalek JJ (2015). "Will Subsidies Drive Electric Vehicle Adoption? Measuring Consumer Preferences in the U.S. and China." *Transportation Research Part A: Policy and Practice*, **73**, 96–112. doi:10.1016/j.tra.2015.01.002.

Hess S, Bierlaire M, Polak JW (2005). "Estimation of Value of Travel-Time Savings Using Mixed Logit Models." *Transportation Research Part A: Policy and Practice*, **39**(2-3), 221–236. doi:10.1016/j.tra.2004.09.007.

Hess S, Palma D (2019). "**apollo**: A Flexible, Powerful and Customisable Freeware Package for Choice Model Estimation and Application." *Journal of Choice Modelling*, **32**, 100170. doi:10.1016/j.jocm.2019.100170.

Huber J, Train K (2001). "On the Similarity of Classical and Bayesian Estimates of Individual Mean Partworths." *Marketing Letters*, **12**(3), 259–269. doi:10.1023/A:1011120928698.

Imai K, van Dyk DA (2005a). "A Bayesian Analysis of the Multinomial Probit Model Using Marginal Data Augmentation." *Journal of Econometrics*, **124**(2), 311–334. doi:10.1016/j.jeconom.2004.02.002.

Imai K, van Dyk DA (2005b). "**MNP**: R Package for Fitting the Multinomial Probit Model." *Journal of Statistical Software*, **14**(3), 1–32. doi:10.18637/jss.v014.i03.

Imai K, van Dyk DA (2026). ***MNP**: Fitting the Multinomial Probit Model*. doi:10.32614/CRAN.package.MNP. R package version 3.1-6.

Joe H (1995). "Approximations to Multivariate Normal Rectangle Probabilities Based on Conditional Expectations." *Journal of the American Statistical Association*, **90**(431), 957–964. doi:10.1080/01621459.1995.10476596.

Kamakura WA, Russell GJ (1989). "A Probabilistic Choice Model for Market Segmentation and Elasticity Structure." *Journal of Marketing Research*, **26**(4), 379–390. doi:10.1177/002224378902600401.

Kass RE, Raftery AE (1995). "Bayes Factors." *Journal of the American Statistical Association*, **90**(430), 773–795. doi:10.1080/01621459.1995.10476572.

Kim SH, Mokhtarian PL (2023). "Finite Mixture (or Latent Class) Modeling in Transportation: Trends, Usage, Potential, and Future Directions." *Transportation Research Part B: Methodological*, **172**, 134–173. doi:10.1016/j.trb.2023.03.001.

Malsiner-Walli G, Frühwirth-Schnatter S, Grün B (2016). "Model-Based Clustering Based on Sparse Finite Gaussian Mixtures." *Statistics and Computing*, **26**(1–2), 303–324. doi:10.1007/s11222-014-9500-2.

McCulloch RE, Rossi PE (1994). "An Exact Likelihood Analysis of the Multinomial Probit Model." *Journal of Econometrics*, **64**(1-2), 207–240. doi:10.1016/0304-4076(94)90064-7.

McFadden D, Train K (2000). "Mixed MNL Models for Discrete Response." *Journal of Applied Econometrics*, **15**(5), 447–470. doi:10.1002/1099-1255(200009/10)15:5<447::AID-JAE570>3.0.CO;2-1.

Meng XL, Wong WH (1996). "Simulating Ratios of Normalizing Constants via a Simple Identity: A Theoretical Exploration." *Statistica Sinica*, **6**(4), 831–860. URL https://www3.stat.sinica.edu.tw/statistica/j6n4/j6n43/j6n43.htm.

Murray I, Adams RP, MacKay DJC (2010). "Elliptical Slice Sampling." In YW Teh, M Titterington (eds.), *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, volume 9 of *Proceedings of Machine Learning Research*, pp. 541–548. PMLR, Chia Laguna Resort, Sardinia, Italy. URL https://proceedings.mlr.press/v9/murray10a.html.

Neal RM (2000). "Markov Chain Sampling Methods for Dirichlet Process Mixture Models." *Journal of Computational and Graphical Statistics*, **9**(2), 249–265. doi:10.1080/10618600.2000.10474879.

Oelschläger L (2026a). *Overcoming Challenges in Modeling Choice Behavior Heterogeneity*. Ph.D. thesis, Bielefeld University, Bielefeld, Germany. URL https://pub.uni-bielefeld.de/record/3014719.

Oelschläger L (2026b). ***choicedata**: Working with Choice Data*. doi:10.32614/CRAN.package.choicedata. R package version 0.2.1.

Oelschläger L (2026c). ***oeli**: Some Utilities for Developing Data Science Software*. doi:10.32614/CRAN.package.oeli. R package version 0.7.8.

Oelschläger L, Bauer D (2021). "Bayes Estimation of Latent Class Mixed Multinomial Probit Models." In *Proceedings of the 100th Annual Meeting of the Transportation Research Board*. Washington, DC. URL https://trid.trb.org/view/1759753.

Oelschläger L, Bauer D (2026). ***RprobitB**: Bayesian Probit Choice Modeling*. doi:10.32614/CRAN.package.RprobitB. R package version 2.0.1.

Papastamoulis P, Iliopoulos G (2010). "An Artificial Allocations Based Solution to the Label Switching Problem in Bayesian Analysis of Mixtures of Distributions." *Journal of Computational and Graphical Statistics*, **19**(2), 313–331. doi:10.1198/jcgs.2010.09008.

R Core Team (2026). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria. doi:10.32614/R.manuals. URL https://www.R-project.org/.

Riphahn RT, Wambach A, Million A (2003). "Incentive Effects in the Demand for Health Care: A Bivariate Panel Count Data Estimation." *Journal of Applied Econometrics*, **18**(4), 387–405. doi:10.1002/jae.680.

Robert CP (1995). "Simulation of Truncated Normal Variables." *Statistics and Computing*, **5**(2), 121–125. doi:10.1007/BF00143942.

Rossi PE (2025). ***bayesm**: Bayesian Inference for Marketing/Micro-Econometrics*. doi:10.32614/CRAN.package.bayesm. R package version 3.1-7.

Rousseau J, Mengersen K (2011). "Asymptotic Behaviour of the Posterior Distribution in Overfitted Mixture Models." *Journal of the Royal Statistical Society B*, **73**(5), 689–710. doi:10.1111/j.1467-9868.2011.00781.x.

Sachs MC (2017). "**plotROC**: A Tool for Plotting ROC Curves." *Journal of Statistical Software, Code Snippets*, **79**(2), 1–19. doi:10.18637/jss.v079.c02.

Sarrias M (2016). "Discrete Choice Models with Random Parameters in R: The **Rchoice** Package." *Journal of Statistical Software*, **74**(10), 1–31. doi:10.18637/jss.v074.i10.

Sarrias M, Daziano R (2017). "Multinomial Logit Models with Continuous and Discrete Individual Heterogeneity in R: The **gmnl** Package." *Journal of Statistical Software*, **79**(2), 1–46. doi:10.18637/jss.v079.i02.

Scarpa R, Thiene M, Marangon F (2008). "Using Flexible Taste Distributions to Value Collective Reputation for Environmentally Friendly Production Methods." *Canadian Journal of Agricultural Economics*, **56**(2), 145–162. doi:10.1111/j.1744-7976.2008.00122.x.

Solow AR (1990). "A Method for Approximating Multivariate Normal Orthant Probabilities." *Journal of Statistical Computation and Simulation*, **37**(3-4), 225–229. doi:10.1080/00949659008811306.

Stephens M (2000). "Dealing with Label Switching in Mixture Models." *Journal of the Royal Statistical Society B*, **62**(4), 795–809. doi:10.1111/1467-9868.00265.

Train KE (2009). *Discrete Choice Methods with Simulation*. 2nd edition. Cambridge University Press, Cambridge. doi:10.1017/CBO9780511805271.

van Kinschot CMJ, Soekhai VR, de Bekker-Grob EW, Visser WE, Peeters RP, van Ginhoven TM, van Noord C (2021). "Preferences of Patients and Clinicians for Treatment of Graves' Disease: A Discrete Choice Experiment." *European Journal of Endocrinology*, **184**(6), 803–812. doi:10.1530/EJE-20-1490.

Van Rossum G, Drake FL (2009). *Python 3 Reference Manual*. CreateSpace, Scotts Valley, CA. ISBN 1441412697.

Vehtari A, Gabry J, Magnusson M, Yao Y, Bürkner PC, Paananen T, Gelman A (2026). ***loo**: Efficient Leave-One-Out Cross-Validation and WAIC for Bayesian Models*. doi:10.32614/CRAN.package.loo. R package version 2.10.1.

Vehtari A, Gelman A, Gabry J (2017). "Practical Bayesian Model Evaluation Using Leave-One-Out Cross-Validation and WAIC." *Statistics and Computing*, **27**(5), 1413–1432. doi:10.1007/s11222-016-9696-4.

Vehtari A, Gelman A, Simpson D, Carpenter B, Bürkner PC (2021). "Rank-Normalization, Folding, and Localization: An Improved $\widehat{R}$ for Assessing Convergence of MCMC (with Discussion)." *Bayesian Analysis*, **16**(2), 667–718. doi:10.1214/20-BA1221.

Vehtari A, Simpson D, Gelman A, Yao Y, Gabry J (2024). "Pareto Smoothed Importance Sampling." *Journal of Machine Learning Research*, **25**(72), 1–58. URL https://www.jmlr.org/papers/v25/19-556.html.

Watanabe S (2010). "Asymptotic Equivalence of Bayes Cross Validation and Widely Applicable Information Criterion in Singular Learning Theory." *Journal of Machine Learning Research*, **11**(116), 3571–3594. URL https://www.jmlr.org/papers/v11/watanabe10a.html.

Wickham H (2016). ***ggplot2**: Elegant Graphics for Data Analysis*. 2nd edition. Springer-Verlag, Cham. doi:10.1007/978-3-319-24277-4.

Zeileis A, Croissant Y (2010). "Extended Model Formulas in R: Multiple Parts and Multiple Responses." *Journal of Statistical Software*, **34**(1), 1–13. doi:10.18637/jss.v034.i01.

Affiliation:

Lennart Oelschläger, Dietmar Bauer
Department of Business Administration and Economics
Bielefeld University
Postfach 10 01 31
33501 Bielefeld, Germany
E-mail: lennart.oelschlaeger@uni-bielefeld.de, dietmar.bauer@uni-bielefeld.de

Journal of Statistical Software http://www.jstatsoft.org/
published by the Foundation for Open Access Statistics http://www.foastat.org/
MMMMMM YYYY, Volume VV, Issue II *Submitted:* yyyy-mm-dd
doi:10.18637/jss.v000.i00 *Accepted:* yyyy-mm-dd